The Scaly Programming Language

The Scaly Programming Language

Table of Contents

1 Introduction **9**
1.1 Overview 9
1.2 Design Philosophy 9
1.3 Implementation Status 9
2 Tutorial **10**
2.1 Getting Started 10
2.1.1 Installing Scaly 10
2.1.2 Hello, World! 11
2.1.3 Installing a Program 12
2.1.4 Trying Things Out 12
2.2 Using Scaly with VS Code 13
2.2.1 Installing the Scaly Extension 13
2.2.2 What the Language Server Gives You 13
2.2.3 Build and Run Tasks 14
2.2.4 Launch Configuration 15
2.2.5 Breakpoints in Scaly Files 16
2.2.6 Setting a Breakpoint and Stepping 16
2.2.7 Stepping into Functions and the Call Stack 17
2.2.8 What the Debugger Shows 17
2.3 A Tour of Scaly 18
2.3.1 Values and Bindings 18
2.3.2 Operators and Operations 18
2.3.3 Control Flow 19
2.3.4 Functions 19
2.3.5 Procedures and Mutation 20
2.3.6 Structures and Methods 20
2.3.7 Unions, Options, and choose 21
2.3.8 Modules and Imports 21
2.4 Putting It Together 22
3 Lexical Structure **23**
3.1 Overview 23
3.2 Whitespace and Line Breaks 23
3.3 Comments 23
3.4 Literals 24
3.4.1 Integer Literals 24
3.4.2 Floating-Point Literals 24
3.4.3 String Literals 24
3.4.4 Character Literals 25
3.4.5 Boolean Literals 25

3.5 Operators and Special Symbols....26
3.5.1 The Equals Sign....26
3.6 Identifiers....26
4 Type System....27
4.1 Overview....27
4.2 Intrinsic Types (Future Direction)....27
4.3 Type Aliases (Future Direction)....28
4.4 Backend Preludes (Future Direction)....28
4.5 Literal Type Inference....29
4.6 Tuples and the Empty Tuple....30
4.7 References....30
4.7.1 Low-Level Pointers....30
4.7.2 Nullable Values....31
4.8 Parameter Passing....31
4.8.1 Implementation....31
4.8.2 Mutation via Procedures....32
4.9 Constructors....32
4.9.1 Implicit Constructors....32
4.9.2 Explicit Constructors....32
4.9.3 The `this` Prefix....33
4.9.4 Complete Initialization....33
4.9.5 Constructor Return....33
4.9.6 Types That Allocate....34
4.10 Lambdas and Closures....34
4.10.1 Capturing Variables....34
4.10.2 Function Types: Delegates....35
4.10.3 Closure Lifetime....35
4.10.4 Implementation....36
4.11 Error Handling....36
4.11.1 The `throws` Clause....36
4.11.2 Single Error Type....37
4.11.3 The `try`/`when` Pattern....37
4.11.4 Error Propagation....38
4.11.5 Error Lifetime....38
4.12 Union Types....38
4.12.1 Memory Layout....39
4.12.2 Pattern Matching with `choose`....39
4.12.3 Option Type....39
4.12.4 Option Optimization....39
4.13 Generics....40
4.13.1 Monomorphization....40
4.13.2 Name Mangling....41

4.14 Lifetimes and Memory Regions 41
4.14.1 Allocation Is Inferred 41
4.14.2 Escape Checking 41
4.14.3 Explicit Lifetime Suffixes 42
4.14.4 By-Value Returns 42
5 Grammar 43
6 Definitions 47
6.1 Overview 47
6.2 Functions, Procedures, and Operators 48
6.3 Definitions 49
6.3.1 Structure Definitions 49
6.3.2 Union Definitions 50
6.3.3 Function Names 50
6.3.4 Operator Definitions 51
6.3.5 Lazy Parameters 51
7 Expressions 52
7.1 Overview 52
7.2 Literals 53
7.2.1 Boolean Literals 53
7.2.2 Integer Literals 53
7.2.3 Hexadecimal Literals 53
7.2.4 Floating-Point Literals 54
7.2.5 String Literals 54
7.2.6 Character Literals 54
7.3 Operations 54
7.3.1 Prefix Operators 55
7.3.2 Operator Chains 55
7.3.3 Application Before Operators 56
7.3.4 Parentheses and Grouping 56
7.3.5 Casts Take the Whole Operation 56
7.3.6 Identifier Shapes 57
7.3.7 Let Bindings 57
7.4 Tuples and Function Values 57
7.4.1 Tuples 58
7.4.2 Applying a Bound Tuple 58
7.4.3 Functions as Values 58
7.5 Comparison Operators 59
7.5.1 Equality 59
7.5.2 Relational Operators 59
7.6 Boolean Operators 60
7.6.1 Boolean Negation 60

7.6.2 Boolean And/Or 60
7.7 Bitwise Operators 61
7.7.1 Bit Shifts 61
7.8 Lambdas 61
7.8.1 Lambda Application 62
7.8.2 Captures 62
7.8.3 Captures in Braced Bodies 62
7.8.4 Embedded Calls 63
7.8.5 Delegate Values 63
7.8.6 Closures 64
7.9 Choose Expressions 64
7.9.1 Choose on Variant Construction 64
7.9.2 Choose with Binding 64
7.9.3 Multiple When Clauses 64
7.10 Optional Types 65
7.10.1 Is Null Check 65
7.10.2 Optional Type Sugar 65
7.10.3 Generic Option Types 65
7.11 Variant Construction 66
7.11.1 Construct Variants 66
7.11.2 Variant Is Expression 66
8 Control Flow 66
8.1 Overview 66
8.2 If Expressions 67
8.2.1 Basic If 67
8.2.2 If with Bindings 67
8.2.3 Else-If Chains without a Final Else 67
8.3 While Loops 68
8.3.1 Basic While 68
8.3.2 While with Accumulation 68
8.4 For Loops 69
8.4.1 Basic For 69
8.4.2 For with Accumulation 69
8.5 Break and Continue 69
8.5.1 Break Statements 69
8.5.2 Continue Statements 70
8.5.3 Nested Loops 70
8.5.4 Labeled Break and Continue 70
8.6 Repeat Loops 71
8.6.1 Repeat and Break Value 71
8.7 Mutable Variables 71
8.7.1 Var and Assignment 71

8.8 Match 72
9 Modules and Packages 72
9.1 Overview 72
9.2 Modules 73
9.3 Packages 73
9.3.1 Packages from a git repository 74
9.3.2 The cache 75
9.3.3 Versions, and what publishing means 76
9.3.4 Starting a package 78
9.3.5 The programs of a package 78
9.4 Imports and Visibility 79
9.4.1 What Is Visible Without an Import 79
9.4.2 Importing a Single Type 80
9.4.3 Importing Everything from a Module 80
9.4.4 When a Type Is Not Visible 81
A. Standard Library and Packages 81
A.1 The Standard Library 81
A.2 Packages Beside It 82
B. Bootstrap History and the Seed 83
C. On the Shoulders of Giants 83
C.1 The Text of a Program 84
C.2 Names and Values 85
C.3 Control Flow 86
C.4 Types and Definitions 87
C.5 Memory 89
C.6 Self-Scaling 89
C.7 Modules and Packages 89
C.8 An Unexpected Ancestor 90
C.9 And What Is Scaly's Own? 90
C.10 The Implementation 92

List of Examples

4-1. LLVM Prelude (excerpt) 28
4-2. JavaScript Prelude (excerpt) 28

1 Introduction

1.1 Overview

Scaly is a programming language designed to be self-scaling: it adapts to the needs of the developer, from simple scripts to complex systems. The language combines a uniform grammar — with no hardcoded operators, since operators are ordinary identifiers — with optional region-based memory management. An operation is read from left to right, the type of the value in hand saying what comes next; how strongly operators bind is stated by the standard library, not by the compiler, and parentheses give explicit control where it is needed.

1.2 Design Philosophy

Scaly follows these core principles:

- *Uniform syntax* — Operators are ordinary identifiers rather than keywords baked into the grammar, so the language and its standard library share one small, regular syntax.
- *Safety without ceremony* — Memory is managed through regions the compiler infers, and an escape checker proves at compile time that no reference outlives its memory; ordinary code carries no annotations.
- *Incremental complexity* — Simple programs stay simple; advanced features are available when needed.

1.3 Implementation Status

This specification documents the language as it is implemented. Scaly is built by a self-hosting compiler written in Scaly itself; the compiler compiles its own source and reproduces itself bit-identically across successive bootstrap stages.

The compiler pipeline — lexer, parser, modeler, planner, and an LLVM-based emitter — is feature-complete. Current implementation status:

- Lexer, parser, modeler, planner, emitter — *implemented*
- LLVM code generation (JIT and ahead-of-time) — *implemented*
- Region-based memory management with inferred allocation and compile-time escape checking — *implemented*
- Self-hosting bootstrap — *achieved* (the compiler compiles itself)
- Source-level debugging (DWARF via `-g`) — *implemented*
- Language server (**scalyls**, written in Scaly) and a VS Code extension — *available* (see the

tutorial)

2 Tutorial

This chapter is a hands-on introduction to Scaly. It gets you from an empty file to a running program, then takes you on a quick tour of the language. Later chapters cover everything here in depth; the goal now is to write and run real code as fast as possible.

2.1 Getting Started

2.1.1 Installing Scaly

This section starts from a blank machine — a fresh Linux installation or a new Mac — and ends with the whole Scaly toolchain installed: the tool, the compiler, the language server, and (in the next section) editor support in Visual Studio Code with syntax highlighting and a source-level debugger.

Scaly is installed by one command. It needs no **sudo** and puts everything under `~/.scaly`:

```
curl -fsSL https://scaly.io/install.sh | sh
```

For macOS 26 on Apple silicon and for Ubuntu 26.04 and newer the installer downloads the programs ready to run, which takes a few seconds. The one thing Scaly asks of your machine is a C compiler: it links the programs you build.

```
; macOS
xcode-select --install

; Ubuntu
sudo apt install clang
```

On any other macOS or Linux the installer builds Scaly on your machine instead. Scaly ships as its own LLVM IR (a *seed*), so that route needs LLVM 21 — **`brew install llvm@21`**, or **`sudo apt install llvm-21 llvm-21-dev lld-21 clang`** — and takes about half a minute; the installer says which of the two it is doing.

The installer takes the newest version of the toolchain. A version that is out does not change afterwards, so a particular one can be asked for by its number and is the same every time:

```
curl -fsSL https://scaly.io/install.sh | SCALY_VERSION=0.2.0 sh
```

The toolchain's number is its own. The packages it brings — the standard library among them —

have theirs, and a program's `package scaly 0.2.0` goes on meaning the standard library of whatever toolchain is installed, as long as that is of the same line (Section 9.3.3).

Either way it puts three commands on your PATH:

- **scaly**, the tool you will use most: **`scaly run`**, **`scaly build`**, **`scaly test`**, **`scaly new`** and **`scaly install`**, and, started without arguments, an interactive session.
- **scalyc**, the compiler by itself, with the command line of a C compiler (`-o`, `-c`, `-S`, `-O2`).
- **scalyls**, the language server that powers the editor features in Section 2.2.

The standard library and the standard packages — `http`, `https`, `tls`, `json`, `compress`, `pg`, `redis` and `h3` — are installed with them, as sources; a package is compiled the first time a program uses it and kept in `~/.scaly/cache`.

Open a new terminal afterwards so the updated PATH takes effect; then all three work from any directory, as the rest of this tutorial assumes. Running the installer again replaces the toolchain with the current one. To uninstall, remove `~/.scaly` and the PATH line the installer added to your shell profile.

2.1.2 Hello, World!

A Scaly program is just a file of statements — there is no `main` function to declare and nothing to import. Put this single line in a file called `hello.scaly`:

```
print("Hello, World!")
```

Run it:

```
scaly run hello.scaly
```

The program prints:

```
Hello, World!
```

`scaly run` compiles the file in memory and runs it at once; nothing is left on disk. To get a program you can keep and hand to someone else, build it:

```
scaly build hello.scaly -o hello
./hello
```

Add `--release` for the optimised build: the whole program — your code, the standard library and every package it uses — is optimised as one piece.

`scaly new hello` writes such a start for you: a directory `hello` with `hello.scaly` in it — a function, a test for it (**`scaly test hello.scaly`** runs every function whose name begins with `test`) and the line that prints.

That is the whole edit-compile-run loop. Everything that follows is just more interesting things to put between the braces.

2.1.3 Installing a Program

A program somebody wrote in Scaly and keeps in a git repository is one command away. **dazzle**, for one — a DSSSL processor that turns a DocBook book into HTML or a typeset PDF, and the program this documentation is built with:

```
scaly install github.com/rschleitzer/dazzle dazzle 0.1.0
```

The three arguments are the repository, the package in it and the package's version. **scaly** fetches the package and the packages it uses, builds its programs optimised, and puts them into the `bin` directory of your Scaly installation, which the installer put on your PATH (if it is not there, **scaly install** says so). A fourth argument names one program where a package offers several. This needs **git**.

```
scaly install               ; what has been installed so
scaly uninstall dazzle      ; remove one of them
```

Section 9.3.5 has the other side: how a package offers programs.

2.1.4 Trying Things Out

scaly without arguments starts an interactive session. An expression shows its value and its type; a `let` or `var` keeps a value for the entries that follow:

```
$ scaly
scaly> 6 * 7
it: int = 42
scaly> let name "Scaly"
name: String = "Scaly"
scaly> :where name
  name: String
    the value   8 bytes, on the session page of its entry
    its text    5 bytes, on a page of its own
scaly> :quit
```

`:where` shows where a value lives — Scaly manages memory in regions, and here you can

watch it do so. **`:help`** lists the other commands.

2.2 Using Scaly with VS Code

You can drive the whole edit-compile-run-debug loop from Visual Studio Code. Two extensions turn it into a full Scaly IDE:

- The *Scaly* extension provides syntax highlighting and connects the editor to the **scalyls** language server you installed in Section 2.1.1 — giving live error squiggles, hover, go-to-definition, completion, rename, and more.
- The *CodeLLDB* extension provides the debugger. Debugging uses the DWARF information the compiler emits with the `-g` flag, so breakpoints land on real source lines and scalar local variables show their values.

If you do not have VS Code yet, download it from `code.visualstudio.com` (macOS and Linux installers are provided; on Ubuntu, **`sudo snap install code --classic`** also works).

2.2.1 Installing the Scaly Extension

The Scaly extension is distributed as a `.vsix` package on scaly.io. Download it and install it from the command line:

```
curl -fsSLO https://scaly.io/downloads/scaly-vscode.vsix
code --install-extension scaly-vscode.vsix
```

Alternatively, install it from within VS Code: open the Extensions view, click the ⋯ menu in its title bar, choose Install from VSIX…, and pick the downloaded file.

The extension launches **scalyls** from your PATH, so install the toolchain first (Section 2.1.1). If VS Code was already running when you installed the toolchain, restart it (not just the window) so its process picks up the updated PATH. Should the server binary live somewhere unusual, point the `scaly.server.path` setting at it.

2.2.2 What the Language Server Gives You

Open any folder, create a file `double.scaly`, and type a small program:

```
function double(n: int) returns int
    n * 2

let d double(21)
if d = 42
```

```
print("doubled!")
```

As you type, the language server is at work. Things to try:

- *Diagnostics* — misspell a call (say, change `double(21)` to `doubel(21)`): a red squiggle appears and the Problems panel shows `function not found: doubel`. Fix it and the squiggle clears.
- *Hover* — rest the mouse on `double` or a binding to see its signature or type.
- *Go to Definition* (**F12**) and *Find All References* (**Shift**+**F12**) — work across files and into the standard library.
- *Completion* — names in scope and members appear as you type.
- *Rename* (**F2**) — renames a symbol everywhere it is used, shadow-aware.
- *Outline and symbols* — the Outline view and **Cmd**+**Shift**+**O** list the file's functions and types; **Cmd**+**T** searches symbols across the workspace. (Use **Ctrl** instead of **Cmd** on Linux.)
- *Inlay hints* — inferred types appear inline after bindings; *signature help* shows parameter hints while you type a call; *semantic highlighting* and *folding* come for free.

If the server ever needs a kick — say, after you rebuilt it — run Scaly: Restart Language Server from the Command Palette.

2.2.3 Build and Run Tasks

NOTE 1 Because the installed scalyc wrapper sets SCALY_HOME for you, the compiler finds the prelude (which defines `print` and friends) and the standard library no matter where your .scaly files live — you do not have to work inside the compiler's own source tree. The tasks below set `cwd` to `${workspaceFolder}` only so the compiled executable lands beside your source.

Create `.vscode/tasks.json` with a build task that compiles the open file with debug info, and a run task that executes the result:

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "scaly: build (debug)",
            "type": "shell",
            "command": "scalyc",
            "args": ["-g", "-o", "${fileBasenameNoExtension}", "${file}"],
            "options": { "cwd": "${workspaceFolder}" },
            "group": { "kind": "build", "isDefault": true },
            "problemMatcher": {
                "owner": "scaly",
```

```
                "fileLocation": ["autoDetect", "${workspaceFolder}"],
                "pattern": {
                    "regexp": "^(.*):(\\d+):(\\d+):\\s+(error|warning):\\s+(.*)$",
                    "file": 1, "line": 2, "column": 3,
                    "severity": 4, "message": 5
                }
            }
        },
        {
            "label": "scaly: run",
            "type": "shell",
            "command": "${workspaceFolder}/${fileBasenameNoExtension}",
            "options": { "cwd": "${workspaceFolder}" },
            "dependsOn": "scaly: build (debug)"
        }
    ]
}
```

Press **Cmd**+**Shift**+**B** (or **Ctrl**+**Shift**+**B**) to build the file in the active editor. The `problemMatcher` turns any compiler diagnostic into a clickable entry in the Problems panel. Use Terminal > Run Task and pick scaly: run to build and execute.

2.2.4 Launch Configuration

Debugging needs one more extension: *CodeLLDB* (extension id `vadimcn.vscode-lldb`, installable normally from the Extensions view), which provides the `lldb` debugger type used below. Reload the window afterwards (Command Palette > Developer: Reload Window) so the extension activates; if **F5** reports configured debug type 'lldb' is not supported, CodeLLDB is not installed or not enabled.

Create `.vscode/launch.json` so **F5** builds the open file and starts it under the debugger:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Debug Scaly file",
            "type": "lldb",
            "request": "launch",
            "preLaunchTask": "scaly: build (debug)",
            "program": "${workspaceFolder}/${fileBasenameNoExtension}",
            "args": [],
            "cwd": "${workspaceFolder}"
        }
    ]
```

```
}
```

2.2.5 Breakpoints in Scaly Files

VS Code only lets you set breakpoints in files whose language an installed extension has declared debuggable. The Scaly extension from Section 2.2.1 does exactly that for `.scaly` files, so with it installed, clicking in the editor gutter places a breakpoint (a red dot appears) — nothing to configure.

NOTE 2 If you skip the Scaly extension, .scaly files are treated as plain text and the gutter does nothing. In that case you can lift the restriction by hand in .vscode/settings.json:

```
{
    "debug.allowBreakpointsEverywhere": true
}
```

2.2.6 Setting a Breakpoint and Stepping

Put this program in `area.scaly` in your workspace folder. It computes a rectangle's area from two mutable bindings — small, but enough to stop and inspect some values:

```
var width 8
var height 5
var area width * height
if area = 40
    print("the area is 40")
else
    print("unexpected area")
```

Click in the gutter to the left of the `var area width * height` line to set a breakpoint (a red dot appears), then press **F5**. The program builds and runs, and execution stops at that line before it executes — at which point `width` and `height` already have values but `area` does not yet.

With execution paused you can:

- Read `width` and `height` in the Variables pane, or hover them in the editor to see their values.
- Step one line at a time with **F10** (Step Over) — after stepping past the breakpoint line, `area` appears with the value `40`. Step into a called function with **F11**.
- Press **F5** again to continue to the program's end.

Breakpoints, stepping, and variable inspection all rely on the `-g` debug information; without it the program still runs, but the debugger has no source lines to stop on.

2.2.7 Stepping into Functions and the Call Stack

Stepping becomes more interesting once a function is involved. **F10** (Step Over) runs a call as a single step; **F11** (Step Into) descends into the called function so you can watch it run. Put this program in `rectangle.scaly` in your workspace folder — it factors the area calculation out into a function:

```
function area(w: int, h: int) returns int
{
    var result w * h
    result
}

var width 8
var height 5
var a area(width, height)
if a = 40
    print("the area is 40")
else
    print("unexpected area")
```

Set a breakpoint on the `var a area(width, height)` line and press **F5**. When execution stops there, `width` and `height` show in the Variables pane. Now press **F11** instead of **F10**: the editor jumps into the body of `area` and stops on `var result w * h`.

The Call Stack pane (bottom-left of the Run and Debug view) now lists two frames — `area` on top and `main` below it — showing exactly how you got here. Click a frame to switch the Variables pane and the editor to that frame's context: selecting `main` shows `width` and `height` again, while `area` shows its own local `result` (still unassigned at this point, just as `area` was before its line ran in the previous example).

From inside the function you can keep stepping, press **Shift**+**F11** (Step Out) to return to the caller, or **F5** to continue to the end. The function-and-procedure model is covered in Section 2.3.4 and Section 2.3.5.

2.2.8 What the Debugger Shows

Debug support tracks the parts of the language that are most useful to inspect today, and is being widened over time. At present:

- Breakpoints land on top-level statements, bindings, and function bodies.
- The Variables pane shows `var` locals — integers and booleans, and also structures, unions, and pointers with their fields expandable — for whichever stack frame is selected.
- Breakpoints and stepping work line-by-line inside loop and conditional bodies, so you can stop

on individual statements within a `while`, `for`, `if`, or `choose` body, not just on the enclosing statement.

- Function parameters, `let` (immutable) bindings, and some standard-library types such as `String` are not shown yet.
- The Watch view and Debug Console evaluate expressions as C/C++ and do not understand Scaly names. To read a value, use the Variables pane or hover over the name in the editor.

2.3 A Tour of Scaly

This section sweeps quickly across the language using small, complete programs. If something is not fully explained here, do not worry — every feature is covered in detail in the rest of this specification, and each part of the tour points you to the relevant chapter.

2.3.1 Values and Bindings

Bind a value to a name with `let`. There is no equals sign — the name is followed directly by its value. Use `var` for a binding you intend to change, and `:=` to change it. A semicolon starts a comment.

```
let answer 42        ; immutable binding
var count 0          ; mutable binding
count := count + 1 ; assignment is :=
```

The equals sign = is reserved for comparison, so it never gets confused with assignment. See Chapter 3 for the full lexical rules.

2.3.2 Operators and Operations

Operators are ordinary identifiers, not keywords baked into the grammar. An operation is read from left to right; when operators are chained, they combine by the binding strength the standard library gives them — multiplication binds tighter than addition, comparison weaker than both:

```
; 3 + 4 * 5 is 3 + 20, which is 23 — not 35
let result 3 + 4 * 5
if result = 23
    print("got 23")
else
    print("unexpected")
```

Parentheses group explicitly, and an operator written where a value is expected acts as a prefix (`-x`, `not done`, `a - -2`). A call takes its arguments before the operators around it combine,

and a call without parentheses takes exactly one argument: `print "hi"` is `print("hi")`, but an argument that is itself a call takes its parentheses, `print(describe(x))`. See Chapter 7 for the complete model, tuples and functions as values included.

2.3.3 Control Flow

`if` chooses between branches. Single-statement branches need no braces; the condition needs no parentheses:

```
let n 7
if n < 0
    print("negative")
else if n = 0
    print("zero")
else
    print("positive")
```

`while` repeats as long as its condition holds. Use braces for a multi-statement body:

```
var sum 0
var i 1
while i <= 10
{
    sum := sum + i
    i := i + 1
}
; sum is now 55
```

`for` iterates. `for i in 4` runs `i` from 0 up to — but not including — 4:

```
var total 0
for i in 4
    total := total + i
; total is 0 + 1 + 2 + 3 = 6
```

2.3.4 Functions

A function declares its parameter types and return type. The value of the last expression is returned automatically — no `return` keyword is needed for the final result. A single-expression body needs no braces:

```
function add(a: int, b: int) returns int
    a + b
```

```
let s add(3, 4)      ; s is 7
```

Functions can call themselves. Here is the classic factorial:

```
function factorial(n: int) returns int
    if n <= 1
        1
    else
        n * factorial(n - 1)

let f factorial(5)   ; f is 120
```

Functions are pure: they read their arguments and return a value, with no side effects. To mutate, use a procedure.

2.3.5 Procedures and Mutation

A `procedure` is like a function but may change its inputs and perform side effects. The distinction is visible at every call site, so a reader always knows which calls can mutate state. See Section 4.8.2 for the details.

```
define Counter(value: int)
{
    procedure increment(this)
        value := value + 1

    function get(this) returns int
        value
}

var c Counter(0)
c.increment()
c.increment()
let v c.get()        ; v is 2
```

2.3.6 Structures and Methods

Define a structure by listing its fields. A structure with public fields gets a constructor that takes them in order. Methods are functions or procedures whose first parameter is `this`:

```
define Point(x: int, y: int)
{
    function distance_squared(this) returns int
        this.x * this.x + this.y * this.y
```

```
}

let p Point(3, 4)
let d p.distance_squared()   ; d is 25
```

Section 4.9 covers constructors, including explicit `init` forms with their own signatures.

2.3.7 Unions, Options, and choose

A `union` holds one of several variants. Each variant may carry a payload. The most common union is `Option`, which represents a value that may be absent:

```
define Option union(Some: int, None)

let present Option.Some(42)
let absent  Option.None
```

Match on a union with `choose`. Each `when` clause names a binding, then the variant, then the result; the binding is bound to the variant's payload. An `else` clause handles the remaining variants:

```
function or_zero(o: Option) returns int
    choose o
        when v: Some
            v
        else
            0

let a or_zero(Option.Some(7))   ; a is 7
let b or_zero(Option.None)      ; b is 0
```

See Section 4.12 for union layout, the `T?` shorthand, and pattern matching in full.

2.3.8 Modules and Imports

Split a program across files with `module`: the statement `module shapes` loads the sibling file `shapes.scaly`. A type defined there stays hidden until the using file imports it with `use`:

```
; shapes.scaly
define Box(width: int, height: int)
{
    function area(this) returns int
        width * height
}
```

```
; myprog.scaly
module shapes
use myprog.shapes.Box        ; or: use myprog.shapes.*

let b Box(3, 4)
if b.area() = 12
    print("area is 12")
```

Forgetting the import is caught at compile time, and the error states the exact `use` line to add. The standard library needs no imports — it is always visible.

Code that several programs share is a *package*, and a package somebody else wrote is one line away:

```
package json 0.1.3 "github.com/someone/json"
```

scaly fetches it from that git repository the first time the program is run, tested or built; and **`scaly install`** builds the programs such a package offers and puts them on your PATH. See Chapter 9 for packages and their sources, visibility rules, and glob imports.

2.4 Putting It Together

Let us combine the pieces into one small, complete program: a safe division that returns an `Option` instead of crashing on a zero divisor, with the caller deciding what to print.

Start with the function. It returns `None` when the divisor is zero, and `Some` with the quotient otherwise:

```
define Option union(Some: int, None)

function safe_divide(a: int, b: int) returns Option
    if b = 0
        Option.None
    else
        Option.Some(a / b)
```

Now call it and react to the result with `choose`. The `Some` arm receives the quotient; the `else` arm handles division by zero:

```
let result safe_divide(20, 4)
choose result
    when q: Some
        if q = 5
```

```
            print("20 / 4 = 5")
        else
            print("unexpected quotient")
    else
        print("cannot divide by zero")
```

Put both pieces in `divide.scaly`, then build and run it:

```
scaly build divide.scaly -o divide
./divide
```

It prints:

```
20 / 4 = 5
```

Change the call to `safe_divide(7, 0)` and the same program prints `cannot divide by zero` — the union makes the failure a value you must handle, not a crash you can forget.

From here, the rest of this specification fills in the details: the Chapter 3 rules, the full Chapter 7 model, and the Chapter 4 system including generics and the inferred region-based memory management.

3 Lexical Structure

3.1 Overview

This chapter describes the lexical structure of Scaly: how source text is divided into tokens. The lexer transforms a sequence of characters into a sequence of tokens that the parser then processes.

3.2 Whitespace and Line Breaks

Whitespace characters (space, tab) separate tokens but are otherwise insignificant. Line breaks, however, are syntactically meaningful in Scaly — they can terminate statements and declarations.

A colon (`:`) can be used in place of a line break when multiple statements should appear on a single line.

3.3 Comments

Scaly supports two forms of comments:

• *Single-line comments* start with a semicolon (`;`) and extend to the end of the line.

• *Multi-line comments* start with `;*` and end with `*;`.

```
; This is a single-line comment

;* This is a
   multi-line comment *;
```

3.4 Literals

3.4.1 Integer Literals

Integer literals represent whole numbers. They can be written in decimal or hexadecimal notation.

```
42        ; decimal
0x2A      ; hexadecimal (same value)
```

3.4.2 Floating-Point Literals

Floating-point literals represent decimal numbers with a fractional part or an exponent.

```
3.14159       ; pi
2.5e10        ; scientific notation
1.0e-5        ; small number
```

3.4.3 String Literals

String literals are enclosed in double quotes. The supported escape sequences are `\"`, `\\`, `\'`, `\n`, `\r`, `\t`, `\0`, and `\uXXXX` (four hex digits, encoded as UTF-8).

```
"Hello, World!"
"Line 1\nLine 2"      ; with newline escape
"Tab:\tValue"         ; with tab
"Quote: \"nested\""   ; escaped quotes
"\u0041"            ; Unicode escape - the letter A
```

Strings can span multiple lines without any special syntax — newlines are simply included as part of the string content:

```
"This is a string
that spans multiple
lines."
```

A backtick inside a string embeds an expression: the string is built from its parts, and each embedded value is written in the text form of its type — a `String`, a `char`, the bytes of a `Slice[u8]`, an integer, a `bool`, or a floating-point number as the shortest text that reads back to it. A value of another type needs a conversion inside the backticks, and `\`` writes a backtick itself.

```
let n 42
let x 0.1
"The answer is `n`, not `n + 1`, and x is `x`"
                      ; The answer is 42, not 43, and x is 0.1
```

3.4.4 Character Literals

A character is written as a string literal of one character wherever a `char` is expected — a `char` parameter, a typed binding, or the cast `as char`. Escapes count as one character.

```
let a: char "a"        ; lowercase a
let nl "\n" as char    ; newline character
```

3.4.5 Boolean Literals

Boolean literals are `true` and `false`.

```
true
false
```

3.5 Operators and Special Symbols

3.5.1 The Equals Sign

Unlike many programming languages, Scaly reserves the equals sign (=) exclusively for equality comparison. It is *not* used for variable declarations or assignments.

```
; Equality comparison
if x = 42 { ... }         ; tests if x equals 42

; Variable declaration (NO equals sign)
let x 42                  ; declares x with value 42
let y: i32 100            ; with type annotation

; Assignment (its own sign)
x := 43                 ; assigns 43 to x
```

This design eliminates the classic confusion between assignment and comparison that leads to bugs in C-style languages (`if (x = 42)` vs `if (x == 42)`).

NOTE 3 Because = is just an operator like any other, the standard library could define a unary = operator that returns its argument unchanged. This would allow the syntax `let x = 42` to parse as `let x (= 42)`, for those who prefer traditional declaration style.

3.6 Identifiers

Identifiers name variables, functions, types, and other entities. Scaly supports three forms of identifiers:

• *Alphanumeric identifiers* start with a letter and continue with letters, digits, or underscores.

• *Operator identifiers* consist of operator characters such as `+`, `-`, `*`, `/`.

• *String identifiers* are enclosed in single quotes and can contain any characters.

```
myVariable        ; alphanumeric
calculate_total   ; with underscore
++                ; operator identifier
'my identifier'   ; string identifier (with spaces)
```

The form of an identifier does not by itself decide what it names. A variable, a function, or an operator may carry any of the three forms; an operator-character name such as ++ can name an

ordinary function, and a string identifier such as `'divided by'` can name an operator. What a name denotes is fixed by its declaration, not by how it is spelled (see the chapter on definitions).

4 Type System

4.1 Overview

Scaly currently recognizes a fixed set of primitive types — integers, floats, booleans, and pointer types — which the LLVM-based emitter maps to the corresponding LLVM types. A possible future direction, described in the following sections, is to make the type system fully backend-agnostic, with even the primitive types supplied by a backend-specific prelude rather than built into the compiler.

4.2 Intrinsic Types (Future Direction)

NOTE 4 The mechanism described in this section and the two that follow (*Type Aliases* and *Backend Preludes*) is a planned design, not current behavior. Today the compiler knows its primitive types directly and targets LLVM only.

An *intrinsic type* is a type whose implementation is provided by the backend, not by Scaly code. Under this design, intrinsic types would be declared using the `intrinsic` keyword:

```
define i32 intrinsic
define i64 intrinsic
define f64 intrinsic
```

The compiler would not interpret these definitions — it would simply record that these types exist and are intrinsic. The backend (Emitter) would be responsible for mapping intrinsic types to the target platform's representation.

This design would enable Scaly to target any backend:

• *LLVM*: `i32`, `i64`, `f32`, `f64`

• *JavaScript*: `number`, `string`, `boolean`

• *WebAssembly*: `i32`, `i64`, `f32`, `f64`

• *JVM*: `int`, `long`, `float`, `double`

4.3 Type Aliases (Future Direction)

Type aliases provide human-friendly names for intrinsic or compound types:

```
define bool i1
define char i32       ; Unicode scalar value
define int i64        ; Platform word size
define size_t u64
```

The alias and its target are interchangeable — `bool` and `i1` refer to the same type.

4.4 Backend Preludes (Future Direction)

Each backend provides a *prelude* file that defines the intrinsic types and their aliases for that target. The prelude is implicitly loaded before any user code.

Example 4-1. LLVM Prelude (excerpt)

```
; Intrinsic types (LLVM native)
define i1 intrinsic
define i8 intrinsic
define i16 intrinsic
define i32 intrinsic
define i64 intrinsic
define f32 intrinsic
define f64 intrinsic
define ptr intrinsic

; Human-friendly aliases
define bool i1
define char i32
define int i64
define size_t u64
```

Example 4-2. JavaScript Prelude (excerpt)

```
; Intrinsic types (JavaScript native)
define number intrinsic
define string intrinsic
define boolean intrinsic
```

```
; Aliases for compatibility
define int number
define float number
define bool boolean
```

4.5 Literal Type Inference

Numeric and other literals do not have an inherent type. Their type is inferred from context — the parameter a literal is passed to, the annotation on its binding, or the other operand of an operation:

```
function double(x: i32) returns i32 { x * 2 }

double(42)           ; 42 inferred as i32 from parameter type

let y: i64 100       ; 100 inferred as i64 from annotation
let z y + 50         ; 50 inferred as i64 to match y
```

When nothing constrains an integer literal, it defaults to `int` (the platform word size), so the common case needs no annotation:

```
let x 42             ; x is int
```

An integer constant — a literal, a `define` declared `int`, and an operation over such — keeps its exact value until it meets the type it is used as, and then it must fit that type. A constant that does not fit is a compile-time error, never a value cut to the width:

```
let a: u8 200 + 55   ; 255
let b: u8 300        ; error: the constant 300 does not fit u8
let c: u32 0 - 1     ; error: the constant -1 does not fit u32
let d: u8 ~0         ; 255: the complement in the target's width
```

Scaly does *not* support type suffixes on literals (such as `42i32`). Where a different width matters, annotate the binding (`let x: i32 42`) or cast (`42 as i32`).

4.6 Tuples and the Empty Tuple

A tuple is an anonymous record, written in parentheses: `(1, "one")`, or with labels, `(x: 1, y: 2)`. Its type is written the same way, `(int, String)`, and it may stand wherever a type may: as a parameter, a result, a field or a generic argument. A labelled component is read by its label, `t.x`; an unlabelled one is reached by decomposition, `let (a, b) t`. A routine returns several values as a tuple:

```
function divide(n: int, d: int) returns (int, int)
    (n / d, n % d)

let (q, r) divide(17, 5)     ; q is 3, r is 2
```

A parenthesized single value is that value, so `(a)` is `a`. The empty tuple `()` is the type of no value: a routine that answers nothing returns `()`, which the word `void` also names. It holds no bytes, so `Result[(), Error]` or `Vector[()]` cost nothing beyond their other parts. A call that answers `()` where a type is declared is a compile-time error.

4.7 References

By default, values flow through functions by value: a function parameter gets read-only access to its argument, and a function returns a value.

To refer to an existing value without copying it, Scaly provides the reference type `ref[T]` — a non-owning, borrowed reference to a `T`. References are used for parameters and for nullable fields, written `ref[T]?`. Like `pointer[T]`, a reference is represented as a machine pointer, but it carries borrow semantics rather than the unchecked, low-level meaning of a raw pointer.

4.7.1 Low-Level Pointers

For low-level implementation of data structures like `Page`, `HashMap`, or `List`, the type `pointer[T]` is available. This is an escape hatch for implementors, not for everyday code.

```
; Low-level list node (internal implementation)
define Node[T]
(
    data: T
    next: pointer[Node[T]]    ; raw pointer for linked structure
)
```

4.7.2 Nullable Values

For nullable values, use `Option[T]` from the standard library — a proper sum type with `Some(value)` and `None` variants, matched with `choose`:

```
let o Option[int].Some(7)
choose o
    when v: Some
        print("got a value")
    else
        print("empty")
```

For many element types the compiler optimizes `Option[T]` to a simple nullable pointer — no space overhead for the tag (see Section 4.12.4).

4.8 Parameter Passing

All *function* parameters are *borrowed* — functions receive read-only access to their arguments. The caller retains ownership.

```
define Point ( x: int, y: int )

function distance_squared(a: Point, b: Point) returns int
{
    let dx a.x - b.x
    let dy a.y - b.y
    dx * dx + dy * dy
}

let origin Point(0, 0)
let target Point(3, 4)
distance_squared(origin, target)    ; origin and target unchanged
```

4.8.1 Implementation

The implementation of parameter passing depends on the execution context, but the semantics remain identical:

- *Same thread/stack*: A pointer is passed. No copying occurs. The function reads through the pointer.

- *Different thread/GPU/remote*: The entire data tree is copied to the target execution context. The function still has read-only access — same semantics, different mechanism.

This design means code doesn't change based on where it executes. A function that works locally works identically when distributed.

4.8.2 Mutation via Procedures

To modify data, use a `procedure` instead of a function. A procedure's parameters may be mutated, and the caller sees the change — no extra keyword is needed; the `procedure` declaration itself is the signal:

```
define Point ( x: int, y: int )

procedure move(p: Point, dx: int)
    p.x := p.x + dx

var pos Point(0, 0)
move(pos, 5)          ; pos.x is now 5
```

The distinction between functions (pure, read-only) and procedures (may mutate) is explicit in the code. Readers immediately know which calls might have side effects.

4.9 Constructors

Constructors create instances of types. Scaly provides both implicit and explicit constructors.

4.9.1 Implicit Constructors

If all members of a type are public, an implicit constructor is generated that takes all fields as parameters in declaration order:

```
define Point ( x: int, y: int )

let p Point(10, 20)    ; implicit constructor
```

4.9.2 Explicit Constructors

Use `init` for explicit constructors when you need default values for some fields or different construction signatures. An explicit `init` coexists with the implicit all-fields constructor:

```
define Point
(
    x: int
    y: int
)
{
    init(value: int)        ; convenience constructor
    {
        this.x := value
        this.y := value
    }
}

let p1 Point(10, 20)    ; implicit constructor
let p2 Point(5)         ; convenience init - Point(5, 5)
```

4.9.3 The `this` Prefix

The `this.` prefix is optional when unambiguous, but recommended for clarity:

```
init(x: int, y: int) {
    this.x := x    ; clear: field x gets parameter x
    this.y := y
}
```

When parameter names shadow field names, `this.` is required to disambiguate. Avoid `x := x` — it's confusing even if technically resolvable.

4.9.4 Complete Initialization

Every constructor should initialize all fields. The compiler does not yet enforce this: a field an `init` forgets to set contains an undefined value, not an implicit default (no automatic 0, false, or null). Enforcement as a compile-time error is planned; until then, treat a missing assignment in an `init` as a bug.

4.9.5 Constructor Return

`init` implicitly returns `this`, enabling direct binding:

```
let p Point(10, 20)    ; init returns the new Point
```

4.9.6 Types That Allocate

Some types are value types (the struct itself can live on the stack) but allocate internal data — `String`, `Array`, and the other containers. Their constructors are ordinary `init` definitions; the region their internal data is allocated in is chosen by the compiler from how the constructed value is used (see Section 4.14). No special constructor syntax is involved.

4.10 Lambdas and Closures

A lambda is an anonymous function defined with backslash syntax — parameters, a colon, and a body:

```
\x: x * 2                    ; single parameter
\x y: x + y                  ; multiple parameters
```

A lambda bound with `let` is called like a named function. Its parameter types are inferred at each call site (the lambda is specialized per call, like a local generic function):

```
let inc \x: x + 1
inc(41)                      ; 42
```

4.10.1 Capturing Variables

Lambdas can capture variables from their enclosing scope. Captures are *by value*: the lambda receives a copy at the call, so later changes to the original are not seen by the closure, and an assignment to a captured name inside the lambda changes only the closure's copy — the original stays untouched:

```
let multiplier 10
let scale \x: x * multiplier    ; captures multiplier by value

scale(5)    ; returns 50
```

4.10.2 Function Types: Delegates

To pass a lambda as a value — to a parameter, into a field, or as a return value — the function type must be named. A `delegate` definition supplies that name:

```
define IntFn delegate(int) returns int

function apply(f: IntFn, x: int) returns int
    f(x)

let g: IntFn \x: x + 1
apply(g, 5)                  ; 6
```

A delegate-typed value is a closure: a record holding the captures plus the code to run. Calling through it is an indirect call.

A named function becomes such a value where a delegate type is declared, chosen among its overloads by the delegate's signature; so does a method bound to its receiver, written without its tuple:

```
function twice(n: int) returns int
    n * 2

apply(twice, 21)             ; 42
let c Counter(40)
apply(c.plus, 2)             ; c.plus is the method bound to c
```

Bound to a name without a delegate type, a function stays a compile-time value: `let g twice` costs nothing, and `g(21)` is a direct call.

4.10.3 Closure Lifetime

A closure bound to a local variable lives on the stack frame and dies with it. A closure *returned* from a function must outlive the frame, so the compiler allocates it on the caller's region automatically — factory functions work:

```
define IntFn delegate(int) returns int

function make_adder(n: int) returns IntFn
{
```

```
    return \x: x + n          ; allocated on the caller's region
}

let add5 make_adder(5)
let r add5(2)                 ; 7
```

What is *not* allowed is smuggling a stack-allocated closure out of its frame — returning a local binding, storing it into an object that outlives the frame, or passing it to a callee that stores it. The compiler rejects these with a hard error:

```
function bad(n: int) returns IntFn
{
    let g: IntFn \x: x + n
    return g
    ; error: a closure bound to a local lives on the stack frame and
    ; cannot be returned - return the lambda directly (return \x: ...)
    ; so it is allocated on the caller's region
}
```

4.10.4 Implementation

A named-called lambda costs nothing at runtime: each call site is monomorphized into a direct call, with captures passed as leading arguments. A delegate-typed closure is a small record — a code pointer plus the captured values — and calling it is one indirect call. No garbage collector is involved either way.

4.11 Error Handling

Scaly uses explicit error handling. There are no exceptions or stack unwinding — errors are values returned from functions or procedures.

4.11.1 The `throws` Clause

Functions or procedures that can fail declare their error type with `throws`:

```
function parse(input: String) returns AST throws ParseError
{
    if invalid(input)
        throw ParseError.InvalidSyntax(String("expected expression"))
    ...
```

```
}
```

Under the hood, this is equivalent to returning `Result[AST, ParseError]`, but with dedicated syntax for clarity.

4.11.2 Single Error Type

A function can only throw one error type. To represent multiple error kinds, use a union:

```
define FileError union
(
    NotFound: String
    PermissionDenied: String
    IoError: String
)

function read_file(path: String) returns String throws FileError
{
    ...
}
```

This ensures a clear error signature and enables the `try`/`when` pattern for handling specific variants.

4.11.3 The `try`/`when` Pattern

Handle errors with `try` and `when` clauses. On success, the binding holds the result and the `when` arms are skipped; on failure, the matching arm runs with the error value bound:

```
try let v read_thing(false)
    when e: NotFound
        print("not found")
    else
        print("other error")
```

If not all error variants are covered by `when` arms, an `else` clause is required.

4.11.4 Error Propagation

Use `else throw` to re-throw errors to the caller:

```
function process(x: int) returns int throws E
{
    try let v inner(x)
        else throw    ; re-throws E to the caller

    v * 2
}
```

When the error type matches exactly, the error propagates automatically. This simplifies deeply nested code like parsers and visitors.

4.11.5 Error Lifetime

Thrown values live in an exception region the caller provides. The compiler infers this from throw position; the explicit thrown-lifetime suffix `!` may also be written on the thrown construction:

```
throw ParseError.InvalidSyntax(msg)!   ; explicit thrown lifetime
```

4.12 Union Types

A union type (also called sum type or tagged union) can hold one of several variants. Each variant optionally carries a payload of a single type (use a struct for several fields):

```
define Rectangle ( width: f64, height: f64 )

define Shape union
(
    Circle: f64
    Box: Rectangle
    Empty
)

let s Shape.Circle(5.0)
```

4.12.1 Memory Layout

A union is stored as a tag plus the largest variant:

```
Shape = { tag: u8, data: [size of largest variant] }
```

Unions can contain other unions. Nested unions contribute their full size (tag + data) when computing the parent union's size.

4.12.2 Pattern Matching with `choose`

Use `choose`-`when` to match on variants. Each `when` clause names a binding, then the variant, then the result; the binding is bound to the variant's payload:

```
choose s
    when r: Circle: r * r * 3.14
    when b: Box: b.width * b.height
    else 0.0
```

If not all variants are covered, an `else` clause is required.

4.12.3 Option Type

`Option[T]` is the standard library's union for nullable values:

```
define Option[T] union
(
    Some: T
    None
)

let o Option[int].Some(7)
choose o
    when v: Some: process(v)
    else handle_not_found()
```

4.12.4 Option Optimization

For non-pointer types, `Option[T]` optimizes to a pointer:

• `None` = null pointer

• `Some(value)` = pointer to value

Note that if T is itself a pointer, the Option Optimization leads to a pointer to another pointer which could be null itself.

4.13 Generics

Scaly supports generic types with type parameters in square brackets:

```
define Pair[T] ( first: T, second: T )
{
    function sum(this) returns T
        first + second
}

let p Pair[int](20, 22)
p.sum()                      ; 42
```

The standard library's containers are generic the same way: `Vector[T]`, `List[T]`, `HashMap[K, V]`, `Option[T]`, `Result[T, E]`.

4.13.1 Monomorphization

Generics are implemented via *monomorphization*: each concrete instantiation becomes a completely separate type at compile time. `Pair[int]` and `Pair[String]` share no code at runtime — each has its own specialized implementation.

Benefits:

• No runtime overhead — no type descriptors or vtables

• Full optimization — the compiler sees concrete types

• No boxing — primitives stay primitives

Trade-offs:

• Larger binaries — each instantiation duplicates code

• Longer compile times — more code to generate

4.13.2 Name Mangling

The Planner generates unique mangled names for each instantiation, following Itanium ABI conventions:

```
List[int]            → _ZN4ListIiE...
List[String]         → _ZN4ListI6StringE...
HashMap[String, int] → _ZN7HashMapI6StringiE...
```

These names are compatible with `c++filt` for debugging.

4.14 Lifetimes and Memory Regions

Scaly uses Region-Based Memory Management (RBMM). Instead of garbage collection or manual malloc/free, values that need the heap are allocated in memory regions (pages) that are deallocated in bulk when their owning scope exits.

4.14.1 Allocation Is Inferred

Ordinary code carries no memory-management annotations at all. A constructed value lives on the stack when that suffices; when its internals allocate, or the value must outlive the statement that created it, the compiler places the allocation in an appropriate region:

• A value used only within the current function gets a *local* region, freed when the scope exits.

• A value returned from a function is placed in the *caller's* region, so it survives the return.

• A thrown error value is placed in the exception region the caller provides.

The standard library and the Scaly compiler itself are written in exactly this annotation-free style.

4.14.2 Escape Checking

The compiler proves that no reference outlives the memory it points into. A reference to a local that would escape the frame — through a return, through a store into an object that outlives the frame, or by being passed to a callee that stores it — is a hard compile-time error:

```
function dangling() returns pointer[int]
{
    var local 5
    return &local
}
```

```
; error: reference into a local page escapes via return - allocate the
; target with '#' (caller page) or '^name' (region) so it outlives the use
```

The check is interprocedural (it follows the value through calls into callees that store their parameters, and through closures) and conservative: code it accepts is safe.

4.14.3 Explicit Lifetime Suffixes

When a value must be placed in a region the inference would not choose — most commonly a heap object returned by pointer — an explicit lifetime suffix on the constructor call says where it lives. The suffix goes between the type and the argument list:

Caller (`#`)

> The value is allocated in the caller's region. It survives the function return and is freed with the caller's scope. Used for returning heap-allocated values by pointer.

Reference (`^name`)

> The value is allocated in the same region as the named value — for tying an allocation's lifetime to a specific longer-lived object, such as a collection it is stored into.

Thrown (`!`)

> The value is allocated in the exception region. The compiler infers this for values in throw position, so writing it is rarely necessary.

```
define Node ( data: int )

function create_node(data: int) returns pointer[Node]
    Node#(data)              ; allocated in the caller's region

let n create_node(7)
(*n).data                    ; 7 — valid in the caller's scope
```

4.14.4 By-Value Returns

Functions can return values by value — the simplest and most common case, and the reason most code needs no lifetime suffix at all:

```
function create_point(x: int, y: int) returns Point
{
    Point(x, y)    ; constructed and returned by value
}
```

By-value return avoids region allocation entirely. The value is constructed directly in the caller's stack frame or register.

5 Grammar

This chapter presents the complete grammar of the Scaly programming language in a formal notation. Each production defines a syntactic construct, with nonterminals linking to their definitions.

Notation:

• *identifier*, *literal* — terminal symbols from the lexer

• `'keyword'` — keyword terminals

• `X | Y` — alternatives

• `X?` — optional

• `X+` — one or more

• `X*` — zero or more

Program = *File* *Statement*`*`

File = *Package*`*` *Use*`*` *Declaration*`*`

Declaration = *Symbol* `':'?`

Symbol = *Private* | *Definition* | *Function* | *Operator* | *Trait* | *Macro* | *Module*

Private = `'private'` *Export*

Export = *Definition* | *Function* | *Operator* | *Trait* | *Module*

Member = *Constituent* `':'?`

Constituent = *Definition* | *Function* | *Procedure* | *Operator* | *Implement* | *Trait* | *Macro* | *Module*

Definition = `'define'` `IDENTIFIER` *GenericParameters*`?` `':'?` *Attribute*`*` *Concept*

GenericParameters = `'['` *GenericParameter*`+` `']'`

GenericParameter = `IDENTIFIER` *Attribute** `','`

Concept = *Class* | *Namespace* | *Union* | *Constant* | *Delegate* | *Intrinsic*

Class = *Structure* `':'?` *Body*`?` `':'?`

Body = `'{'` *Use** *Init** *DeInit*`?` *Member** `'}'` `':'?`

Namespace = `'{'` *Use** *Declaration** `'}'` `':'?`

Union = `'union'` `':'?` `'('` *Variant*+ `')'` `':'?` *Body*`?` `':'?`

Variant = `IDENTIFIER` *Attribute** *TypeAnnotation*`?` `','` `':'?`

Constant = *Type* *Operand** `':'?`

Delegate = `'delegate'` *ParameterSet*`?` *Attribute** *Returns*`?` *Throws*`?`

GenericArguments = `'['` *GenericArgument** `']'`

GenericArgument = *Type* `','`

Optional = `'?'`

ParameterSet = *Parameters* | *Type*

Parameters = `'('` *Item** `')'`

Item = `IDENTIFIER` *TypeAnnotation*`?` `','` `':'?` *Attribute** `':'?`

Returns = `'returns'` *Type* *Attribute**

Throws = `'throws'` *Type* *Attribute**

Use = `'use'` *Name* `':'?`

Implement = `'implement'` *Type* *Attribute** `':'?` `'{'` *Use** *Method** `'}'` `':'?`

Trait = `'trait'` *Name* *Extends*`?` *Attribute** `'{'` *Use** *Method** `'}'` `':'?`

Method = *Function* | *Procedure* | *Operator*

Init = `'init'` *ParameterSet*`?` `':'?` *Action* `':'?`

DeInit = `'deinit'` `':'?` *Action* `':'?`

Function = `'function'` *Target*

Procedure = `'procedure'` *Target*

Operator = `'operator'` *Target*

Target = *Named* | *Routine*

Named = `IDENTIFIER` *Routine*

Routine = *GenericArguments*`?` *Lifetime*`?` *ParameterSet*`?` *Attribute*`*` *Returns*`?` `':'?` *Throws*`?` `':'?` *Implementation*

Implementation = *Action* | *Extern* | *Instruction* | *Intrinsic*

Extern = `'extern'`

Instruction = `'instruction' ':'?`

Intrinsic = `'intrinsic' ':'?`

Extends = `'extends'` *Extend*`*`

Extend = *Type* `','`

Macro = `'macro' IDENTIFIER` *Model* *Operand*+

Attribute = `ATTRIBUTE` *Model* `':'?`

Model = *Literal* | *Name* | *Object* | *Vector*

Module = `'module' IDENTIFIER ':'?`

Package = `'package'` *Name* *Version*`?` *Source*`?` `':'?`

Version = `LITERAL '.' LITERAL`

Source = `LITERAL`

Initializer = `'('` *Operand*+ `')'`

Operand = *Expression* *MemberAccess*`*`

MemberAccess = `'.'` *Name*

Expression = *Literal* | *Type* | *Object* | *Vector* | *Block* | *If* | *Match* | *Lambda* | *For* | *While* | *Choose* | *Try* | *Repeat* | *SizeOf* | *Is*

Literal = `LITERAL`

Object = `'('` *Component*`*` `')'`

Component = *Operand*+ *Attribute*`*` *Value*`?` `','`

Value = `':'` *Operand*+ *Attribute*`*`

Vector = `'['` *Element*+ `']'` *Lifetime*`?`

Element = *Operand*+ *Attribute*`*` `','`

Block = `'{'` *Use*`*` *Statement*`*` `'}'`

If = `'if'` *Operand*+ `':'` *Command* *Else*?

Else = `'else'` `':'`? *Command*

Match = `'match'` *Operand*+ `':'` *Branch*+ *Else*?

Branch = *Case*+ `':'` *Statement*

Case = `'case'` *Operand*+

Statement = *Command* `':'`?

While = `'while'` *Condition* `':'` *Label*? *Action*

Choose = `'choose'` *Operand*+ `':'` *When** *Else*?

Try = `'try'` *Condition* `':'` *When** *Else*?

Condition = *Operation* | *Let*

When = `'when'` `IDENTIFIER` `':'` *Name* `':'`? *Command* `':'`?

Command = *Operation* | *Let* | *Var* | *Mutable* | *Set* | *Continue* | *Break* | *Return* | *Throw*

Let = `'let'` *Binding*

Var = `'var'` *Binding*

Mutable = `'mutable'` *Binding*

Binding = `IDENTIFIER` *BindingAnnotation*? *Operand*+

BindingAnnotation = `':'` *BindingSpec*

BindingSpec = *Structure* | *Type* | *Array*

Array = `'['` *Type** `']'`

Structure = `'('` *Part** `')'`

Part = *Field* | *Property*

Field = `'private'` *Property*

Property = `IDENTIFIER` *TypeAnnotation* *Initializer*? `','` `':'`? *Attribute** `':'`?

TypeAnnotation = `':'` *Type*

Continue = `'continue'` *Loop*? `':'`?

Break = `'break'` *Loop*? *Operand** `':'`?

Loop = `'loop'` `IDENTIFIER`

Return = `'return'` *Operand**

Throw = `'throw'` *Operand**

Lambda = `'lambda'` *Operand*+ `':'` *Action*

For = `'for'` `IDENTIFIER` *TypeAnnotation*`?` `'in'` *Operand*+ `':'` *Label*`?` *Action*

Label = `'label'` `IDENTIFIER` `':'``?`

Repeat = `'repeat'` *Label*`?` *Action*

Action = *Operation* | *Set*

Operation = *Operand*+

Set = `'set'` *Operand*+ `':'` *Operand*+

SizeOf = `'sizeof'` *Type*

Is = `'is'` *Name*

Type = *Name* *GenericArguments*`?` *Optional*`?` *Lifetime*`?`

Name = `IDENTIFIER` *Extension**

Extension = `'.'` `IDENTIFIER`

Lifetime = *Call* | *Local* | *Reference* | *Thrown*

Call = `'#'`

Local = `'$'`

Reference = `'^'` `IDENTIFIER`

Thrown = `'!'`

6 Definitions

6.1 Overview

Definitions declare new types that can be used throughout a program. Types can have properties, methods, and nested definitions.

6.2 Functions, Procedures, and Operators

A routine is introduced by one of three keywords, and that keyword — not the spelling of the name — decides how the routine behaves:

- *function* declares a pure function: its inputs are borrowed and immutable, and it returns a result without side effects. A function is applied prefix, by writing its name before a parenthesized argument list.
- *procedure* declares a routine that may mutate its inputs and have side effects (builders, I/O).
- *operator* declares an operator: a pure routine applied infix between its two operands (or prefix, for a unary operator).

The three identifier forms from the lexical chapter — alphanumeric, operator-character, and single-quoted string identifiers — are interchangeable as names here. A function may be named with operator characters, and an operator may be named with letters or a quoted string; what makes one a function and the other an operator is the declaration keyword, never the first character of the name.

```
function 'add seven'(n: int) returns int      ; string-named function
    n + 7

function ++(n: int) returns int               ; operator-char-named function
    n + 1

let a 'add seven'(35)                          ; applied prefix -> 42
let b ++(41)                                   ; applied prefix -> 42
```

Operators may be declared at the top level, not only as members of a type. Like functions, they accept any of the three name forms:

```
operator <+>(left: int, right: int) returns int    ; operator-char name
    left + right

operator 'divided by'(left: int, right: int) returns int   ; quoted name
    left / right

let c 40 <+> 2              ; infix -> 42
let d 84 'divided by' 2     ; infix -> 42
```

Built-in arithmetic, comparison, and logical operators carry their usual precedences. A custom

operator that no level of the operation names — a text or single-quoted name, or a symbol such as == — binds most weakly of all, on a level of its own below `||`, so the built-in arithmetic in an expression combines before it. For example, given an operator `'minus'` that subtracts its right operand from its left, `20 'minus' 3 + 5` parses as `20 'minus' (3 + 5)` and yields `12`.

An operator's right parameter may be declared `lazy`: the caller's operand is handed over unevaluated, and the operator's body is put in where the operator stands. The standard library declares `and` and `or` on `bool` this way, so the right operand runs only where the left one does not decide:

```
operator and(left: bool, right: lazy bool) returns bool
    if left: right else false

let ok p <> null and p.x = 7   ; p.x is read only where p is there
```

The right operand is planned where the body puts it, so a null test on the left proves its name for it, as an `if` proves it for its branch. A free function and a method may declare a lazy parameter too: the call is the body put in where it stands, ordinary arguments evaluated once in the order they are written, a lazy one where the body names it:

```
function or_else(v: int, fallback: lazy int) returns int
    if v <> 0: v else fallback

let n or_else(cached, expensive())   ; expensive() runs only where cached is 0
```

6.3 Definitions

Definitions declare new types that can be used throughout a program.

6.3.1 Structure Definitions

The define keyword creates a new structure type with named properties.

Input	Result
`define Point(x: int, y: int)` `Point(3, 4)`	`{"_type":"Point","x":3,"y":4}`
`define Point(x: int, y: int)` `Point(3, 4).x`	`3`

Input	Result

`define Point(x: int, y: int)`
`let p Point(3, 4)`
`p.x + p.y`
	`7`

`define Point(x: int, y: int)`
`define Rectangle(origin: Point, width: int, height: int)`
`Rectangle(Point(0, 0), 10, 5).width`
	`10`

6.3.2 Union Definitions

Unions define types with multiple variants. Each variant can optionally carry a value.

Input	Result
`define Option union(Some: int, None)` `Some(5)`	`{"_type":"Option","_variant":"Some","value"`
`define Option union(Some: int, None)` `None`	`{"_type":"Option","_variant":"None"}`
`define Result union(Ok: int, Err: str)` `Ok(42)`	`{"_type":"Result","_variant":"Ok","value":4`
`define Result union(Ok: int, Err: str)` `Err("error")`	`{"_type":"Result","_variant":"Err","value":`
`define Option union(Some: int, None)` `Option.Some(5).value`	`5`

6.3.3 Function Names

A routine's role is set by its declaration keyword, not by how its name is spelled. A function may be named with ordinary letters, with a single-quoted string identifier (which may contain spaces), or with operator characters; in every case it is an ordinary function, applied by writing its name before a parenthesized argument.

Input	Result

`function 'add seven'(n: int) returns int`
`    n + 7`
`'add seven'(35)`
	`42`

`function ++(n: int) returns int`
`    n + 1`
`++(41)`
	`42`

6.3.4 Operator Definitions

The operator keyword declares an operator, applied infix between its two operands. Like functions, operators may be named with operator characters or with a text or single-quoted name. Operators may be declared at the top level, not only inside a type. An operator that no level of the operation names, a text-named one among them, binds more weakly than every level, so the arithmetic combines first.

Input	Result

```
operator <+>(left: int, right: int) returns int
    left + right
40 <+> 2
```

```
42
```

```
operator 'plus'(left: int, right: int) returns int
    left + right
40 'plus' 2
```

```
42
```

```
operator 'divided by'(left: int, right: int) returns int
    left / right
84 'divided by' 2
```

```
42
```

```
operator 'minus'(left: int, right: int) returns int
    left - right
20 'minus' 3 + 5
```

```
12
```

6.3.5 Lazy Parameters

A parameter declared `lazy` before its type receives its argument unevaluated: the routine's body is put in where the call stands, and the argument runs where the body names it, and only there. This is how the standard library declares `and` and `or`, and it works for an ordinary function as well.

Input	Result

```
function or_else(v: int, fallback: lazy int) returns int
    if v <> 0: v else fallback
procedure loud() returns int
{
    print "evaluated"
    7
}
or_else(5, loud())
```

```
5
```

```
function or_else(v: int, fallback: lazy int) returns int
    if v <> 0: v else fallback
or_else(0, 6 * 7)
```

```
42
```

7 Expressions

7.1 Overview

Expressions are the core computational units of Scaly. Every expression evaluates to a value; one that answers nothing answers the empty tuple `()`. An expression is an *operation*: a sequence of things — values, names, operator names, tuples, vector literals and blocks — that the compiler reads from left to right. It holds one current value, and the *type* of that value says what the next thing means:

current value	next thing	meaning
none, a prefix, or one waiting for its operand	an operator name	a prefix: it takes the next value (`-x`, `a - -2`, `not not a`)
a value	an operator its type declares	that operator; it waits for its right operand
a function	a tuple	the call; the tuple is the argument list
a function	one other value	the call with that one argument (`print "hi"`, `twice -5`)
a type	a tuple	the construction (`Point(1, 2)`)
a type	a vector literal	the generic instance (`Vector[int]`)
a value	a vector literal	indexing (`v[i]`)
any	`as T`, `is T`	the cast or the test of the whole operation so far

Parentheses make a tuple (and a tuple of one value is that value, so they also group), brackets a vector literal, braces a block, and the dot a member. Nothing else structures a sequence: there is no precedence table in the compiler. How strongly operators bind is stated by the standard library, as a protocol of *pending* types: `a +` answers a value that waits for its right operand, and the protocol decides what an operator after it means. The levels, strongest first, are listed under Section 7.3.

Two consequences are worth knowing before reading the rest of this chapter:

- *Application is stronger than waiting.* Before an operator takes what follows, that thing takes what is applied to it: `a + f(1)` adds the call's result, `a + v[i]` the element.
- *A call without parentheses takes exactly one thing.* `print f(x)` is `(print f)(x)`, which the compiler reports; an argument that is itself a call, an indexing or a construction takes its parentheses: `print(f(x))`.

7.2 Literals

Expressions are the core computational units of Scaly. Every expression evaluates to a value.

7.2.1 Boolean Literals

The boolean literals are true and false.

Input	Result
`true`	`true`
`false`	`false`

7.2.2 Integer Literals

Integer literals are sequences of decimal digits, optionally preceded by a minus sign for negative numbers.

Input	Result
`42`	`42`
`0`	`0`
`-7`	`-7`

7.2.3 Hexadecimal Literals

Hexadecimal literals start with 0x or 0X followed by hexadecimal digits (0-9, a-f, A-F).

Input	Result
`0xFF`	`255`
`0x1A`	`26`
`0x0`	`0`

7.2.4 Floating-Point Literals

Floating-point literals contain a decimal point separating the integer and fractional parts.

Input	Result
`3.14`	`3.14`
`2.0`	`2.0`
`0.5`	`0.5`

7.2.5 String Literals

String literals are enclosed in double quotes. They may span multiple lines, with line breaks preserved in the resulting string.

Input	Result
`"hello"`	`hello`
`""`	

7.2.6 Character Literals

A character is written as a string literal of one character wherever a char is expected: a char parameter, a typed binding, or the cast `as char`.

Input	Result
`"a" as char`	`a`
`"Z" as char`	`Z`

7.3 Operations

An operation is a sequence of things: values, names, operator names, tuples in parentheses, vector literals in brackets and blocks in braces, each perhaps followed by a member access with a dot. The compiler reads the sequence from left to right and holds one current value; the TYPE of that value says what the next thing means. A tuple after a function is the call, a tuple after a type the construction, a vector literal after a value the indexing, an operator name after a value one of the operators its type declares. Parentheses, brackets, braces and the dot are what structure a sequence, and nothing else does. An operation ends with a colon, a line break, or anything that is not part of it.

7.3.1 Prefix Operators

Where a value is expected an operator name is a prefix: it takes the next value as its operand. A value is expected at the start of an operation, after an operator, after a function, and after another prefix, so `a - -2` subtracts minus two, `twice -5` is `twice(-5)`, and `not not a` reads as written. An = at the start of an operation does nothing, so `let a = 42` reads as `let a 42`.

Input	Result
`-7`	`-7`
`-(-3)`	`3`
`+ 3`	`3`
`10 - -2`	`12`
`- -2`	`2`
`not not true`	`true`

`function twice(n: int) returns int`
`  n * 2`
`twice -5`
	`-10`

`= 6 * 7`	`42`

7.3.2 Operator Chains

Several operators in a row combine by their binding strength, which the standard library states as a protocol of pending types: the compiler itself has no precedence table. Strongest first, the levels are `* / % div mod`; `+ -`; `<< >>`; `< > <= >=`; `= <>`; `&`; `xor`; `|`; `and &&`; `or ||`. An operator a type declares that no level names, such as == or a text name, binds most weakly of all. Operators of one level combine from left to right.

Input	Result
`3 + 4`	`7`
`10 - 3`	`7`
`6 * 7`	`42`
`84 / 2`	`42`
`17 % 5`	`2`
`3 + 4 * 2`	`11`
`20 - 5 - 3`	`12`

Input	Result
`1 << 2 + 1`	`8`
`1 + 2 * 3 = 7`	`true`
`1 < 2 and 3 < 4`	`true`

7.3.3 Application Before Operators

Before an operator takes the thing that follows it, that thing takes what is applied to it: a function its tuple, a value its vector literal, anything its members. So `a + f(1)` adds the call's result, and `a + v[i]` the element. A function called without parentheses takes exactly one thing; an argument that is itself a call, an indexing or a construction takes its parentheses, `print(f(x))`.

Input	Result

`function twice(n: int) returns int`
`    n * 2`
`3 + twice(4) * 2`
	`19`

`function twice(n: int) returns int`
`    n * 2`
`twice 21`
	`42`

7.3.4 Parentheses and Grouping

Parentheses around a single value group it: `(a)` is `a`. The group is read as an operation of its own, so a prefix may begin it inside a longer chain, and it combines first.

Input	Result
`5 * (- 2)`	`-10`
`2 * (3 + 4)`	`14`
`(2 + 3) * (4 + 1)`	`25`

7.3.5 Casts Take the Whole Operation

A cast `as T` and a test `is T` apply to the whole operation before them: `a + b as T` is `(a + b) as T`. A cast of the right side alone takes parentheses.

Input	Result

Input	Result

`let a: u8 200`
`let b: u8 100`
`a + b as int`
	`44`

`let a: u8 200`
`let b: u8 100`
`a + (b as int)`
	`300`

7.3.6 Identifier Shapes

Identifiers can have name shape (alphanumeric) or operator shape (symbols). The shape is purely lexical and does not determine behavior: what a name means depends on its declaration and on what the operation holds when it is reached.

Input	Result
`10 div 3`	`3`
`abs (- 5)`	`5`

7.3.7 Let Bindings

Let bindings introduce local variables. The syntax is let followed by a name and a value expression (no equals sign). Multiple statements are separated by colons or newlines.

Input	Result
`let x 5` `x`	`5`
`let x 5` `x + 3`	`8`

`let x 5`
`let y 3`
`x + y`
	`8`

`let x 2 + 3` `x * 2`	`10`

7.4 Tuples and Function Values

A tuple is a value: an anonymous record written in parentheses. Its components may carry labels, and its type is written the same way, `(int, String)`. The empty tuple `()` is the type of no value, which a routine without a result answers. Functions are values too: a name bound to a

function, or a method bound to its receiver, is called later like the function itself.

7.4.1 Tuples

A labelled component is read by its label. Unlabelled components are reached by decomposition, `let (a, b) t`; a tuple is not an array and has no indexing. A routine returns several values as a tuple.

Input	Result
`let t (x: 3, y: 4)` `t.x * t.y`	`12`
`let (a, b) (6, 7)` `a * b`	`42`

`function pair() returns (int, int)`
`    (3, 4)`
`let (p, q) pair()`
`p + q`
	`7`

7.4.2 Applying a Bound Tuple

A tuple bound to a name stands for its components where it is applied: `f args` calls f with them, and `Point(args)` builds the record from them. Its components are evaluated once, when it is bound.

Input	Result

`function add(a: int, b: int) returns int`
`    a + b`
`let args (40, 2)`
`add args`
	`42`

`define Point(x: int, y: int)`
`let args (1, 2)`
`let p Point(args)`
`p.y`
	`2`

7.4.3 Functions as Values

`let g twice` binds the function at compile time and costs nothing; `g(21)` is a direct call. `x.m` without a tuple is the method bound to its receiver. Where a delegate type is declared, a named function becomes a closure value, chosen by the delegate's signature.

Input	Result

`function twice(n: int) returns int`
`    n * 2`
`let g twice`
`g(21)`
	`42`

`define Counter(n: int)`
`{`
`    function plus(this, k: int) returns int`
`        n + k`
`}`
`let c Counter(40)`
`let m c.plus`
`m(2)`
	`42`

`define IntFn delegate(int) returns int`
`function apply(f: IntFn, x: int) returns int`
`    f(x)`
`function twice(n: int) returns int`
`    n * 2`
`apply(twice, 21)`
	`42`

7.5 Comparison Operators

Comparison operators compare two values and return a boolean result.

7.5.1 Equality

The equality operator (=) tests if two values are equal. The inequality operator (not equal to) tests if two values are different.

Input	Result
`5 = 5`	`true`
`5 = 3`	`false`
`5 <> 3`	`true`
`5 <> 5`	`false`

7.5.2 Relational Operators

Relational operators compare the ordering of values.

Input	Result

Input	Result
`3 < 5`	`true`
`5 < 3`	`false`
`5 > 3`	`true`
`3 > 5`	`false`
`5 <= 5`	`true`
`5 <= 3`	`false`
`5 >= 5`	`true`
`3 >= 5`	`false`

7.6 Boolean Operators

Boolean operators combine boolean values.

7.6.1 Boolean Negation

The word operator `not` negates a boolean value. The tilde is the bitwise complement, which on a bool is the same. The exclamation mark is not negation in Scaly: it is the sigil of the thrown lifetime.

Input	Result
`not true`	`false`
`not false`	`true`
`~true`	`false`
`~false`	`true`

7.6.2 Boolean And/Or

The operators `and` and `or` (also spelled `&&` and `||`) are declared by the standard library with a lazy right operand: the right side is evaluated only where the left one does not decide. The bitwise operators `&`, `|` and `xor` combine two bools as well, and evaluate both sides.

Input	Result
`true and false`	`false`
`false or true`	`true`

Input	Result

`procedure noisy() returns bool`
`{`
`    print "evaluated"`
`    true`
`}`
`false and noisy()`
	`false`

`procedure noisy() returns bool`
`{`
`    print "evaluated"`
`    false`
`}`
`true or noisy()`
	`true`

`true & true`	`true`
`true & false`	`false`
`false | false`	`false`
`true | false`	`true`
`true xor true`	`false`

7.7 Bitwise Operators

Bitwise operators manipulate integer values at the bit level.

7.7.1 Bit Shifts

The left shift operator shifts bits to the left, effectively multiplying by powers of two. The right shift operator shifts bits to the right, effectively dividing by powers of two.

Input	Result
`1 << 4`	`16`
`5 << 3`	`40`
`16 >> 2`	`4`
`100 >> 3`	`12`

7.8 Lambdas

A lambda is an anonymous function written with a backslash: `\params: body`. Parameters are space-separated. A lambda bound to a name and applied to arguments is monomorphized at the

call site for those argument types, so the same lambda works at any argument type.

7.8.1 Lambda Application

A lambda bound to a name can be called like a function.

Input	Result
`let inc \x: x + 1` `inc(5)`	`6`
`let add \x y: x + y` `add(3, 4)`	`7`

`let dbl \x: x + x`
`let a dbl(10)`
`let b dbl(21)`
`a + b`
	`62`

7.8.2 Captures

A lambda captures enclosing variables by value. The captured value is fixed when the lambda is applied.

Input	Result

`let n 10`
`let f \x: x + n`
`f(5)`
	`15`

`let a 3`
`let b 4`
`let g \x: x + a + b`
`g(10)`
	`17`

7.8.3 Captures in Braced Bodies

Capture discovery reaches inside a braced body, and a body-local binding that shadows an enclosing name is the local, not a capture.

Input	Result

`let n 10`
`let cap \x: { x + n }`
`cap(5)`
	`15`

Input	Result

`let n 10`
`let shade \x: { let n 99`
`x + n }`
`shade(5)`
	`104`

7.8.4 Embedded Calls

A lambda call may appear anywhere inside a larger expression, not only as the whole right-hand side of a binding. Like every application, each call takes its arguments before the operators around it combine.

Input	Result

`let inc \x: x + 1`
`let add \x y: x + y`
`inc(5) + add(3, 4)`
	`13`

`let inc \x: x + 1`
`let n 100`
`n + inc(9)`
	`110`

`let n 100`
`let cap \x: x + n`
`cap(1) + cap(2)`
	`203`

7.8.5 Delegate Values

A `delegate` names a function type. A non-capturing lambda assigned to a delegate-typed binding is a function-pointer value that can be passed to a function and called indirectly. A lambda literal may also be passed straight to a delegate parameter, and so may a named function.

Input	Result

`define IntFn delegate(int) returns int`
`function apply(f: IntFn, x: int) returns int`
`    f(x)`
`let g: IntFn \x: x + 1`
`apply(g, 5)`
	`6`

`define IntFn delegate(int) returns int`
`function twice(f: IntFn, x: int) returns int`
`    f(f(x))`
`let dbl: IntFn \x: x + x`
`twice(dbl, 20)`
	`80`

7.8.6 Closures

A capturing lambda assigned to a delegate type is a true closure value carrying the captured variables. (Closures returned from a function escape onto the caller's region -- see the escaping-closure AOT regression test.)

Input	Result

`define IntFn delegate(int) returns int`
`function apply(f: IntFn, x: int) returns int`
`    f(x)`
`let n 10`
`let add: IntFn \x: x + n`
`apply(add, 5)`
	15

7.9 Choose Expressions

Choose expressions pattern match on union types, allowing different code paths based on which variant is present.

7.9.1 Choose on Variant Construction

Choose can match directly on constructed variants.

Input	Result
`define Option union(Some: int, None)` `choose Option.Some(7): when v: Some: v else 0`	7
`define Option union(Some: int, None)` `choose Option.None: when v: Some: v else 0`	0

7.9.2 Choose with Binding

When a variant carries a value, the value can be bound to a name.

Input	Result
`define Result union(Ok: int, Err: int)` `choose Result.Ok(42): when x: Ok: x + 1 else 0`	43

7.9.3 Multiple When Clauses

Choose can have multiple when clauses to match different variants.

Input	Result
`define Result union(Ok: int, Err: int)` `choose Result.Ok(10): when x: Ok: x when e: Err: e + 100`	10
`define Result union(Ok: int, Err: int)` `choose Result.Err(5): when x: Ok: x when e: Err: e + 100`	105

7.10 Optional Types

Optional types use null pointer optimization (NPO) for efficient representation.

7.10.1 Is Null Check

The is null expression checks if an optional value is None.

Input	Result

`define Option[T] union: (Some: T, None)`
`function check(x: int?) returns int: if x is null: 0 else 1`
`check(Option[int].None)`
	0

`define Option[T] union: (Some: T, None)`
`function check(x: int?) returns int: if x is null: 0 else 1`
`check(Option[int].Some(42))`
	1

7.10.2 Optional Type Sugar

T? is syntactic sugar for Option[T].

Input	Result

`define Option[T] union: (Some: T, None)`
`function unwrap(x: int?) returns int: choose x: when v: Some: v else 0`
`unwrap(Option[int].Some(42))`
	42

7.10.3 Generic Option Types

Generic unions like Option[T] work with choose for pattern matching.

Input	Result
`define Option[T] union: (Some: T, None)` `choose Option[int].Some(42): when v: Some: v else 0`	42

Input	Result
`define Option[T] union: (Some: T, None)` `choose Option[int].None: when v: Some: v else -1`	-1

7.11 Variant Construction

Union variants can be constructed explicitly using Type.Variant syntax.

7.11.1 Construct Variants

Variants are constructed with Type.Variant(value) syntax.

Input	Result
`define Result union: (Ok: int, Error: int)` `choose Result.Ok(42): when x: Ok: x else 0`	42
`define Result union: (Ok: int, Error: int)` `choose Result.Error(100): when e: Error: e else 0`	100

7.11.2 Variant Is Expression

The is expression tests if a union value matches a variant.

Input	Result
`define Result union: (Ok: int, Error: int)` `if Result.Ok(50) is Ok : 1 else 0`	1
`define Result union: (Ok: int, Error: int)` `if Result.Error(33) is Ok : 1 else 0`	0

8 Control Flow

8.1 Overview

Conditions and loops are expressions like every other construct: an `if` with both branches has a value, and a `repeat` loop answers the operand of the `break` that ends it. A condition needs no parentheses; a body of one statement needs no braces, a longer one is a block in braces that begins on a line of its own. `if cond stmt` without an `else` runs the statement when the condition is *true*, and control continues after it.

Conditions are written as `and`/`or` chains. A null test proves its name for what follows it in the

chain: in `p <> null and p.x = 7` the right side reads `p.x` only where `p` is there.

8.2 If Expressions

If expressions evaluate a condition and return one of two values.

8.2.1 Basic If

The if expression takes a boolean condition and two branches.

Input	Result
`if true : 1 else 2`	`1`
`if false : 1 else 2`	`2`
`if 5 > 3 : 10 else 20`	`10`

8.2.2 If with Bindings

If expressions can use values from let bindings.

Input	Result
`let x 5` `if x > 3 : x * 2 else x`	`10`

`let a 7`
`let b 3`
`if a > b : a else b`
	`7`

8.2.3 Else-If Chains without a Final Else

An if statement may chain an else if without a final else. Such a chain yields no value; when no condition holds, the statement falls through and any bindings keep their prior values. (Regression: the emitter used to merge the arms' values into a dead phi node that failed LLVM verification when the inner arm's value was computed inside its own branch.)

Input	Result

Input	Result

`function clamp(dmin: int) returns int`
`{`
`    var adj 7`
`    if dmin > 100`
`        adj := 0`
`    else if dmin > 50`
`        adj := dmin + 1`
`    adj`
`}`
`clamp(200) * 10000 + clamp(60) * 100 + clamp(10)`
	6107

8.3 While Loops

While loops repeat as long as a condition is true.

8.3.1 Basic While

A while loop evaluates its body repeatedly while the condition is true.

Input	Result

`var x 0`
`while x < 3 : x := x + 1`
`x`
	3

`var x 10`
`while x < 5 : x := x + 1`
`x`
	10

8.3.2 While with Accumulation

While loops can accumulate values across iterations.

Input	Result

`var sum 0`
`var i 1`
`while i <= 3 : { sum := sum + i: i := i + 1 }`
`sum`
	6

`var result 1`
`var n 5`
`while n > 0 : { result := result * n: n := n - 1 }`
`result`
	120

8.4 For Loops

For loops iterate over a range of integers.

8.4.1 Basic For

A for loop iterates from 0 to n-1 where n is the range expression.

Input	Result

`var count 0`
`for i in 3 : count := count + 1`
`count`
	`3`

`var x 42`
`for i in 0 : x := 0`
`x`
	`42`

8.4.2 For with Accumulation

For loops can accumulate values using the loop variable.

Input	Result

`var sum 0`
`for i in 5 : sum := sum + i`
`sum`
	`10`

`var prod 1`
`for i in 4 : prod := prod * (i + 1)`
`prod`
	`24`

8.5 Break and Continue

Break exits a loop early, continue skips to the next iteration.

8.5.1 Break Statements

Break exits the innermost enclosing loop.

Input	Result

`var sum 0`
`for i in 10 : { if i = 3 : break else sum := sum + i }`
`sum`
	`3`

Input	Result

`var x 0`
`while x < 100 : { x := x + 1: if x = 5 : break }`
`x`
	`5`

8.5.2 Continue Statements

Continue skips the rest of the current iteration and continues with the next.

Input	Result

`var sum 0`
`for i in 6 : { if i % 2 = 1 : continue else sum := sum + i }`
`sum`
	`6`

`var sum 0`
`var i 0`
`while i < 10 : { let current i: i := i + 1: if current % 2 = 1 : continue else sum :`
`sum`
	`20`

8.5.3 Nested Loops

By default break and continue affect only the innermost loop.

Input	Result

`var count 0`
`for i in 3 : { for j in 10 : { if j = 2 : break else count := count + 1 } }`
`count`
	`6`

`var count 0`
`for i in 3 : { for j in 5 : { if j % 2 = 1 : continue else count := count + 1 } }`
`count`
	`9`

8.5.4 Labeled Break and Continue

A loop may carry a label written `label NAME` after its colon. From within a nested loop, `break loop NAME` exits the named loop and `continue loop NAME` jumps to its next iteration, rather than acting on the innermost loop.

Input	Result

`var count 0`
`for i in 3 : label outer { for j in 5 : { if j = 2 : break loop outer else count :=`
`count`
	`2`

Input	Result
`var count 0` `for i in 3 : label outer { for j in 5 : { if j = 0 : continue loop outer else count count`	`0`

8.6 Repeat Loops

A repeat loop runs its body forever and exits only through break (or return). Because break is the only way out, a break carrying a value (`break v`) makes the repeat an expression that yields that value.

8.6.1 Repeat and Break Value

The value of a repeat expression is the operand of the break that exits it.

Input	Result

`var i 0`
`let answer repeat { if i = 5 : break i + 100 else i := i + 1 }`
`answer`
	`105`

`var i 0`
`repeat { if i = 3 : break else i := i + 1 }`
`i`
	`3`

8.7 Mutable Variables

A binding introduced with var may be assigned a new value; one introduced with let may not. The assignment is written `target := source`, in the Algol tradition, and is a statement of its own: it introduces no name and is never part of a condition. The equals sign always compares.

8.7.1 Var and Assignment

Var creates a mutable variable, and `:=` assigns it.

Input	Result
`var x 5` `x`	`5`

`var x 5`
`x := 10`
`x`
	`10`

Input	Result

`var x 3`
`x := x + 1`
`x`
	`4`

`var a 2`
`var b 3`
`a := a * b`
`a`
	`6`

8.8 Match

`match` compares one value against a list of `case` alternatives and takes the first that is equal; one case may name several values, and `else` takes the rest. It works on numbers, characters and strings, which it compares by content:

```
function kind(n: int) returns int
    match n
        case 1: 10
        case 2 case 3: 20
        else 0

let word "b"
match word
    case "a": print "A"
    case "b": print "B"
    else print "?"
```

A union is matched by its variants with `choose`, described under Section 4.12.

9 Modules and Packages

9.1 Overview

A Scaly program can be split across several source files, and it can depend on other packages. There are no build files, manifests, or configuration files — three statements inside the source itself organize everything:

- `module` loads another source file into the current program or package.
- `package` declares a dependency on another package.
- `use` makes types defined in another module visible in the current file.

9.2 Modules

Every source file is a module. The statement `module shapes` loads the file `shapes.scaly` from the directory of the file that declares it and makes it part of the program:

```
; shapes.scaly
define Box(width: int, height: int)
{
    function area(this) returns int
        width * height
}

; myprog.scaly
module shapes
use myprog.shapes.Box

let b Box(3, 4)
if b.area() = 12
    print("area is 12")
```

A module may declare child modules of its own. Those live in a directory named after the parent module: if `compiler.scaly` declares `module lexer`, the child is loaded from `compiler/lexer.scaly`. This mirrors how the standard library and the compiler themselves are laid out.

Top-level statements of a loaded module are ignored — only its definitions become part of the program. Programs have top-level statements; libraries consist of definitions only.

9.3 Packages

A package is a source distribution. Its version is encoded in its filesystem path — `packages/scaly/0.2.0/` — and that path is the single source of truth; there is no separate version manifest. The statement

```
package scaly 0.2.0
```

declares a dependency on that package. A package is looked for among the packages of the installation, `$SCALY_HOME/packages/` (the installed programs set SCALY_HOME themselves), and then among the project's own, `./packages/` under the directory the compiler runs in. So a project brings the packages the installation does not have by putting them beside its sources, and it cannot put a package of its own in front of an installed one of the same name. Without SCALY_HOME only the current directory is searched. A declared package that is nowhere fails the compile with an error naming the places.

The version in a declaration is a *minimum*. One compilation takes one version of a package, and where a program reaches the same package through several declarations — its own and those of the packages it uses — it takes the highest version any of them names: a program that asks for `json 0.1.5` and uses a package that asks for `json 0.1.2` is built, with `0.1.5` for both. That holds among versions that go together — within one major version, and below 1.0 within one second number (Section 9.3.3 says why). Such versions form a *line*. Two lines of one package — a package that asks for `json 0.1` beside one that asks for `json 0.2` — are two packages to the build: both are compiled and linked, each asker gets its own, and a name of `json` written in a file means the line that file's package declares. What does not go: a value of one line handed to a routine of the other (they are different types), one file that declares both lines, and two lines of a package that brings C or assembly files. Each line has its own state — a cache or a counter kept in a package's global exists once per line — and what builds this way is not raised, so **scaly publish** says when a newer line of a dependency exists. Nothing else decides: there is no lock file and no solver, the versions follow from the declarations alone, and adding a dependency can raise a version but never lower one. A package's root file declares the packages it uses itself, so a program names only the packages it calls directly; circular package dependencies are forbidden.

The standard library package `scaly` is the implicit *prelude*: every program depends on it automatically, and its types are visible everywhere without any `use` statement. Pass `--no-prelude` to compile without it, for bare-metal or embedded targets. See Appendix A for what the standard library provides.

9.3.1 Packages from a git repository

A declaration may name where its package comes from, as a string after the version:

```
package json 0.1.3 "github.com/someone/json"
```

The source is a git repository that holds the package at `packages/json/0.1.3/` — the layout a project's own packages have. One repository serves every version it has published (Section 9.3.3). `"github.com/someone/json"` stands for `https://github.com/someone/json.git`; an address with a scheme, a `user@host:path` address and a local path are taken as they are, and whatever git needs to reach a private repository is git's business. A version is required where a source is given.

scaly run, **scaly test** and **scaly build** fetch what a program declares before they compile it, and what the fetched packages declare in their turn; **scalyc** never reaches the network and says which commands do. This needs **git** on the PATH. A fetched package is kept under `~/.scaly/packages/` (SCALY_PACKAGES names another place), by source, name and version, with its files read-only; nothing is written into the project. It is the third place a package is looked for, after the installation and the project's `./packages/` — so copying a package there is how a project keeps its own copy, and editing that copy is how it patches a dependency.

A version the package's record names is taken from the commit named there, any other from the head of the default branch. A published version does not change (Section 9.3.3). The first fetch remembers the commit and the content of the version's directory beside it; a version that is fetched again and is no longer what it was is refused, with both commits named. One package comes from one source: a program that reaches it through two different sources is refused, while two spellings of one address are one source.

A package may bring C or assembly files, which are compiled on the machine that builds it; the tool says so when it fetches one.

What a package defines is the package's own: two packages may each define a function, a record with its methods or a mutable global of one name, and each keeps its own. A record of one package is another type than a same-named record of another, and a name written in a package means that package's own before anything a `use` brings in. A program may hold values of both, call their methods and hand each back to its package; handing one package's record to the other is refused like any wrong type. The standard library's names and a program's own are not marked.

A package's constants and mutable globals are its own as well. A mutable global is one cell wherever it is read: a program that names a package's counter reads the counter the package's routines write. A bare name means the global of the package it is written in before another package's of the same name; written where two packages in reach both declare it and neither is the writer's own, it is refused, since nothing says which is meant.

A `use` makes a name writable — as a type, in a construction, before a static call. A value brings its type along: its methods and fields are reached without naming the type, so `let a make_alpha(1)` and `a.grown()` need no `use` of what `make_alpha` returns.

9.3.2 The cache

A package is compiled once. The first program that uses it has it compiled into the cache, `~/.scaly/cache` (SCALY_CACHE names another place); every later build links what lies there. Beside the compiled package lies its *interface* — the package as a caller sees it, every declaration and no body — and that is what the compiler reads from then on instead of the sources. Nothing of this is written into a project or a package, and nothing of it is ever committed.

Each entry is kept under a key over everything that went into it: the compiler, every file of the package, and the versions of the packages it declares. So an entry is never wrong, only no longer asked for — a changed package or a new compiler leaves the files of the state before. **`scaly clean`** empties the cache; what is needed is made again by the next build. The packages that were fetched (`~/.scaly/packages`) and the programs that were installed (`~/.scaly/bin`) are not the cache and stay.

9.3.3 Versions, and what publishing means

The rules of a package's versions are few, and the tool checks them.

A version is a directory. `packages/json/0.1.3/` is version 0.1.3 of `json`, and the next version is another directory beside it. There are no tags, no registry and no manifest.

Published is what the package's record names. Beside its version directories a package keeps the file `packages/json/published`, a line for each published version:

```
0.1.2 8c1f0e…  5d2a97…
0.1.3 b41c77…  0e93ab…
```

The version, the commit its directory lay at when it was published, and the git tree of that directory. **scaly publish** writes the lines; nobody else needs to. A version directory *without* a line is a version being written: it may change from commit to commit, on the default branch too, and a program that declares it gets it as it stands — and is told when it has changed since.

A published version does not change. Its line stays as it is, and its directory, while it is in the tree, stays the tree the line names. A correction, however small, is a new version: copy the directory, change the copy.

A published version may leave the tree. The line names the commit that holds it, and a build that declares the version fetches that commit. So a repository need not carry every version it ever published side by side: the directories being worked on are in the tree, the others in its history. Removing one is an ordinary commit that deletes the directory and leaves the record alone.

The number says what changed. A new version is compared with the one before it — the highest lower version, in the tree or in the record — by what the two *declare*: every function, procedure, operator and initializer with its parameters and its result, every record with its fields, every union with its variants, every constant, global and alias. Bodies are not part of it, a generic routine's body neither. A package it declares counts too, by its line: a version that moves from `json 0.1` to `json 0.2` has changed what it declares, because a program that uses both packages must follow. Adding or dropping a dependency changes nothing a user had.

against the version before	from 1.0 on	below 1.0
a declaration is gone or reads differently	the first number rises	the second number rises
declarations were added, none is gone	the second number rises	the third number rises
bodies only	the third number rises	the third number rises

A version below 1.0 has promised nothing yet, so there a breaking change costs the second number; 1.0.0 is the statement that what the package declares is meant to stay. A build follows the same reading: the highest version asked for stands in for the lower ones where the table says nothing was taken away — within one first number from 1.0 on, within one second number below it. `0.2.0` does not stand in for `0.1.5`, and a program that reaches both is told so.

The comparison is deliberately strict. Declarations are compared as they are written, comments and line breaks apart: a renamed parameter, a constant's new value, a field added to a record all count as a declaration that reads differently. And nothing in a package is private, so every declaration counts.

```
scaly publish --check
scaly publish
```

Both run at the root of the package's repository and speak with the remote `origin` (another remote, an address or a path may follow). **`scaly publish --check`** changes nothing: it says whether every published version is still what its line names, whether the remote's record is still all here, and whether every new version's number says what it changed. A version that understates its change is refused with the declarations that are gone and the number it takes at least; a new version that does not compile is refused too. Where a new version declares a package of which a newer line has been published meanwhile, it says so in a note — the declaration is not wrong, but no build will take the newer line until its author raises it. It answers 0 when all is well, 1 when a rule is broken and 2 when it could not ask — no repository, or the remote out of reach. It belongs in the package's own test run, so a change to a published version is noticed before it is pushed and not by the first program that fetches it.

`scaly publish` checks the same and then publishes what is new: each new version gets its line — the commit is the current one, so the version must be committed —, the record is committed, and both commits are pushed. From then on the version is fixed.

Making the next version of a package is therefore: give `packages/json/0.1.3/` the number the change takes, work and commit there as long as it takes, and **`scaly publish`** when it is done. The directory may be copied, and then both versions lie side by side; it may as well be *renamed*, because the published one lives on at its commit, and then the tree holds one directory per package. **`scaly next json 0.1.4`**, at the root of the repository, does that: it renames the directory with git and rewrites every tracked file that names it (a script, a test). It wants a tree without uncommitted changes and a number higher than the one that is there, and it leaves a published directory of another package as it was published. A package that is released together with a program — a compiler with its parser, say — publishes a version with each release and works on the next one in between.

A declared version is a minimum. `package json 0.1.3` asks for 0.1.3 or what stands in for it. Among the declarations of one build that is the highest version asked for; and where a place a package is looked for holds no directory of the version itself, it is the highest version of that line

there that is not lower. So the declarations need not follow when a package they name begins its next version: a program that says `package json 0.1.3` builds against a tree or an installation that has `packages/json/0.1.4/` alone. A version of another line never stands in, and neither does a lower one.

9.3.4 Starting a package

```
scaly new shapes --lib
```

creates `shapes/packages/shapes/0.1.0/shapes.scaly`: a package with one function and its test. Its root file declares the packages it uses — the standard library too, which only a program gets without saying so:

```
package scaly 0.2.0

define shapes
{
    function greeting(name: String) returns String
        "Hello, `name`!"

    function test_greeting() returns int
    {
        if greeting("Scaly") <> "Hello, Scaly!"
            return 1
        0
    }
}
```

`scaly test packages/shapes/0.1.0/shapes.scaly` runs its tests. A program in the directory that holds `packages/` uses it with `package shapes 0.1.0`; once the directory is a git repository somewhere, any program does, by naming that repository as the source.

9.3.5 The programs of a package

A package that is a library first may still bring programs — a parser its validator and its normalizer, say. The programs a package offers are the files of the directory `programs/` beside its root file, each a program named after its file:

```
packages/shapes/0.1.0/
    shapes.scaly              the library
    shapes/                   its modules
    programs/
```

```
shapes.scaly          the program "shapes"
shapes_check.scaly    the program "shapes_check"
```

A program there is an ordinary program that declares the package it belongs to, without a source — its package and the others of the same repository lie beside it:

```
package shapes 0.1.0

print(shapes.greeting("world"))
```

One of the programs may be called like the package. Other program files a package keeps — drivers for its tests, experiments — stay outside `programs/` and are not offered.

```
scaly install github.com/someone/shapes shapes 0.1.0
scaly install github.com/someone/shapes shapes 0.1.0 shapes_check
```

fetches the package from that repository, builds its programs — all of them, or the one named — optimised as **`scaly build --release`** does, and puts them beside **scaly** itself, in `~/.scaly/bin` (SCALY_BIN names another place). A package without `programs/` is a library, and **scaly install** says so.

`scaly install` without arguments lists what it has installed — program, package, version and source —, and **`scaly uninstall shapes_check`** removes one of those. It goes by its own record and removes nothing else of that directory.

9.4 Imports and Visibility

Type definitions are private to their module by default. A type defined in another module — even one loaded into the same program with `module` — is hidden until the current file imports it with a `use` statement. This keeps every file's dependencies explicit: the list of `use` statements at the top of a file is a complete inventory of the outside types the file relies on.

9.4.1 What Is Visible Without an Import

Three kinds of types need no `use` statement:

- Types defined *in the same file*.
- Types of the *prelude* — the standard library (`String`, `Vector`, `Option`, …) is always visible.
- The namespace of a *declared child module*. When a file declares `module geometry` and `geometry.scaly` defines a namespace of the same name, that namespace is visible in the declaring file, so its members can be reached qualified:

```
; geometry.scaly
define geometry
{
    function unit_area() returns int
        1
}

; myprog.scaly
module geometry

let a geometry.unit_area()   ; qualified access, no use needed
```

Visibility applies to named type definitions. Free functions are resolved through their namespaces and are not gated separately.

9.4.2 Importing a Single Type

A `use` statement names a type by its full dotted path. The path starts with the program or package name, followed by the module path, ending in the type name:

```
use myprog.shapes.Box            ; Box from module shapes of program myprog
use scalyc.compiler.Modeler      ; Modeler from package scalyc
```

A module that is named after the single type it defines — the file `Modeler.scaly` defining `Modeler` — is addressed once, as the path spells it; the module and the type do not appear as two separate path segments.

9.4.3 Importing Everything from a Module

A trailing `*` imports every type a module defines:

```
module shapes
use myprog.shapes.*

let b Box(3, 4)              ; Box is visible through the glob
if b.area() = 12
    print("glob works")
```

Globs are convenient for a module whose types are used pervasively. Prefer single-type imports when a file uses only one or two names from a module — they document the dependency more precisely.

9.4.4 When a Type Is Not Visible

Referencing a type that exists in the program but has not been imported fails the compile. The error names the module that defines the type and states the exact `use` line that would import it:

```
myprog.scaly:3:7: error: function not found: Box - defined in
    myprog.shapes but not visible here; add: use myprog.shapes.Box
```

Two compiler flags support working with visibility: `--no-use-vis` disables enforcement entirely (every type in the program is visible everywhere, the pre-visibility behavior), and `--use-vis` prints an advisory report of every reference that strict visibility would reject — useful when migrating an existing codebase, to collect the needed `use` lines before turning enforcement on.

A. Standard Library and Packages

A.1 The Standard Library

The standard library is the package `scaly`. Every program sees all of it without a `use` — it is the prelude — unless it is compiled with `--no-prelude`. Its modules:

`scaly.memory`

> The region runtime: pages, regions, the frame pages the compiler gives every routine that allocates, and the traps a program raises on a bounds violation, a null unwrap or a lack of memory.

`scaly.containers`

> `String` and `StringBuilder`; `Slice[T]`, the borrowed view that never owns; `Vector[T]` of a fixed length, `Array[T]` that grows, `List[T]`; `HashMap[K, V]` and `HashSet[T]` with their builders; `Option[T]` (written `T?`) and `Result[T, E]`. Indexing with `[]` traps outside the bounds, `get(i)` answers an Option, and `put(i, v)` writes.

`scaly.fiber`

> Fibers, the scheduler and the evented I/O beneath the operating-system module, the task pool a self-scaling `for` runs on, threads and channels.

`scaly.os`

> `Console`, `File`, `Directory`, `Path` and `Tcp`, and the process environment.

`scaly.cluster`

Channels between processes over TCP: a region sent to another node moves with its whole object graph.

`scaly.tensor`

A tape for reverse-mode differentiation — matrix products, activations, attention, the optimizers — living in one region per tape.

`scaly.operation` and `scaly.logic`

The protocol that gives the operators their binding strength (Chapter 7), generated from the levels of the grammar, and `and` and `or` on `bool`, whose right operand is `lazy`.

A.2 Packages Beside It

These packages come with the toolchain. A program declares the ones it uses (`package json 0.2.0`, see Chapter 9), so a program that never speaks TLS does not link OpenSSL:

`http`, `https`, `h3`

An HTTP/1.1 and HTTP/2 server with a router, static files, WebSocket and gRPC; the same server over TLS 1.3; and HTTP/3 over QUIC (ngtcp2).

`tls`

TLS 1.3 for a TCP stream, bound to OpenSSL, and the hashes and key derivations a protocol above it needs.

`json`

RFC 8259 JSON: a pull reader, a value tree built on it, and a writer.

`compress`

gzip for HTTP bodies, over libdeflate.

`pg`, `redis`, `sqlite`, `tds`

Clients of PostgreSQL (protocol version 3, SCRAM authentication, prepared statements, a pool), Redis (RESP2), SQLite (its own library) and Microsoft SQL Server (TDS 7.4).

`scalygpu`
: A tensor tape resident on the GPU (Metal).

B. Bootstrap History and the Seed

Scaly is self-hosted, and the committed *seed* — the compiler emitted as its own textual LLVM IR under `seed/` — is the sole bootstrap root. The normal way to build the compiler is from that seed — it is what the installer of Section 2.1.1 does on your machine; it needs only LLVM 21 and a C compiler. A single seed covers every 64-bit little-endian (LP64) target, and a CI matrix builds and verifies it natively on macOS and Linux, on both x86-64 and AArch64.

The seed is refreshed by `tools/seed.sh`: it bootstraps a compiler from the committed seed, re-emits the compiler sources with it, and verifies the *fixed point* — the seed must reproduce itself byte-identically — before `tools/install-seed.sh` installs the new files. Textual IR is diffable and auditable, which is why Scaly ships it instead of opaque per-platform binaries. See `RELEASING.md` in the repository for the full seed-distribution checklist.

Historically, the self-hosted compiler was bootstrapped from a reference implementation written in C++ (which had itself replaced an earlier TypeScript bootstrapper). That C++ stage-0 compiler was retired in July 2026, once the seed became self-sufficient: its sources are frozen under `retired/scalyc0/` as a historical reference, it has no build, and it cannot parse the current language. No C++ toolchain appears anywhere in today's build.

Targets with a different data model (32-bit, big-endian, or LLP64 such as Windows) are not covered by the seed; supporting one would mean generalizing the compiler's size and alignment emission off its fixed 64-bit constants and minting a seed for that data model.

C. On the Shoulders of Giants

Hardly anything in Scaly is new. Almost every spelling and almost every mechanism was invented by somebody else, often decades ago, and reached Scaly through a chain of languages that each kept what worked. This appendix follows those chains: for each detail it names the language or the work where the idea appeared first, as far as that can be told, and the languages that carried it on.

Two cautions. A first appearance is hard to prove — where an earlier source turns up, this appendix is wrong and wants correcting. And an entry says where an idea comes from, not that Scaly's author took it from there: many of these ideas arrived second or third hand, and some were found again before their history was looked up.

C.1 The Text of a Program

Comments that begin with `;`

> The semicolon opens a comment in most assembly languages since the 1960s, and in Lisp and its descendants. Scaly has it from Scheme: the compiler's own parser is generated by stylesheets written in DSSSL, which is a Scheme dialect.

Block comments `;*` … `*;`

> The bracketing pair with an asterisk is PL/I's `/*` … `*/` (1964), which B and C made universal. Scaly keeps the shape and exchanges the slash for its own comment sign.

The line as a statement, the colon between two on one line

> That a line break ends a statement is as old as Fortran (1957). Putting a second statement on the same line after a colon is the habit of the BASIC dialects of the 1970s.

Blocks in braces

> Block structure — a group of statements that counts as one and has names of its own — is ALGOL 60's, written `begin` … `end`. CPL bracketed its blocks with the section sign, BCPL with `$(` and `$)`, and B (Ken Thompson, 1969) with the braces that C handed to everybody. Opening each brace on a line of its own is the style named after Eric Allman, whose BSD sources made it known; it keeps the vertical alignment of `begin` and `end`.

Operators are identifiers

> That an operator is an ordinary name made of operator characters, and its meaning an ordinary definition, is the way of ML and Haskell, and of Smalltalk's binary messages; Scala showed that it carries a whole language without a single operator built into the grammar. The words `and`, `or` and `not` are ALGOL 60's logical operators in the spelling ALGOL 68 and Pascal gave them.

= compares, <> is unequal

> The equals sign for equality is mathematics', and ALGOL 60 kept it that way by giving the assignment a sign of its own. <> is how Pascal, BASIC and SQL write ALGOL's ≠ on a keyboard that has none.

Precedence

> The order of the levels — multiplication before addition, then shifts, comparisons, equality, and the bitwise operators below those — is C's table, which C has from B. It is not a table in

the compiler: the levels are written in the standard library as types whose values wait for what follows, and the planner reads an operation by them.

Hexadecimal literals with `0x`

C.

Expressions embedded in a string

Interpolation comes from the Unix shell, where `$name` inside double quotes is replaced by its value; Perl, Ruby and Swift extended it from names to expressions. The backtick as the sign of something to be evaluated in place is the shell's too.

C.2 Names and Values

The assignment `:=`

ALGOL 58, the preliminary report that preceded ALGOL 60, and through ALGOL 60 the whole family: Simula, Pascal, Modula, Ada, Smalltalk in its ASCII form. The sign reads "becomes" and leaves = to mean what it means in mathematics. Before it, Zuse's Plankalkül and Rutishauser's notation wrote an arrow pointing at the target. Go uses the same sign for a declaration; Scaly does not — a name is introduced by `let` or `var`, and `:=` only ever changes one that exists.

`let` and `var`

`let` for a name bound to a value is Peter Landin's ISWIM (1966), and from there ML and every functional language; `var` for something that varies is Pascal's (1970). The pair as two keywords of equal rank, one for the name that stays and one for the name that changes, is Swift's (2014) — Scala and Kotlin write `val` and `var`.

The name first, then a colon, then the type: `count: int`

Parameters, fields, typed bindings and the arms of a union are all declared this way, and among programming languages it is Pascal's way (Niklaus Wirth, 1970). ALGOL 60 and Wirth's own ALGOL W before it put the type first, `integer n`, and C followed them. Older languages had the name in front — COBOL (1959) and PL/I (1964) write a name and then its attributes — but without the colon. The colon itself is older than programming: it is how mathematics states what a thing is, `f: A → B`, and from there it entered type theory (N. G. de Bruijn's Automath, 1967) and ML, independently of Pascal. Nearly every language designed since 2000 has gone back to this order, because a type that may be left out is easier to leave out at the end.

The value of a body is its last expression

> Lisp (1958), where everything is an expression; then ALGOL 68 and BLISS among the block-structured languages, and today Ruby, Scala and Rust. The conditional *expression* itself — an `if` that has a value — is John McCarthy's invention, taken into ALGOL 60.

A call without parentheses

> Application by juxtaposition, `print "x"`, is the notation of the lambda calculus and of ML and Haskell; Logo and Ruby brought it to imperative code. That the argument list in parentheses is a tuple — one value — is ML's view of a function of several arguments.

Integer and float types named by their width

> `i32`, `u8`, `f64` are Rust's names, which follow LLVM's. `size_t` is C's. The vector types `f64x4`, `u8x16` are named as in Rust's portable SIMD and in WebAssembly.

C.3 Control Flow

`while`, `repeat`, `for … in`

> `while` began as a clause of ALGOL 60's `for` statement and became a statement of its own in ALGOL W, BCPL and Pascal. `repeat` is Pascal's word; the unconditional loop it stands for in Scaly is BCPL's `REPEAT`. A `for` that walks over the elements of a collection instead of counting comes from CLU's iterators (1975); the spelling with `in` is the Bourne shell's and Python's.

`break`, `continue`, named loops

> `break` is BCPL's and B's, `continue` C's. Leaving an outer loop by its name is Ada's (`exit Outer`), taken up by Java's labelled `break`. A loop that yields the value its `break` carries is Rust's `loop`.

`match` and `case`

> The case statement was proposed by Tony Hoare and appeared in ALGOL W (1966); C's `switch` is its best-known child.

A self tail call is a loop

> Scheme (Steele and Sussman, 1975) was the first language to require that a call in tail position take no stack, and so made recursion a legitimate way to write a loop.

C.4 Types and Definitions

A structure declared with its fields in parentheses, constructed by a call

> Simula 67: a class is declared with parameters, the parameters are its attributes, and creating an object is handing over their values. Most languages after it separated the fields from the constructor that fills them and then spent decades removing the repetition again — Scala's class parameters (2004), Kotlin's primary constructor (2011), the records of C# and Java. Scaly's `define Point (x: int, y: int)` with `Point(1, 2)` is that form, and like Scala, Kotlin and Swift it needs no `new`.

`init` and `this`

> `init` as the name of the routine that initializes an object is Objective-C's convention, made a keyword by Swift. `this` is Simula's, through C++. Writing the receiver as the first parameter of a method follows Python's `self` and Rust's.

`function` and `procedure`

> The two words are ALGOL's and Pascal's, where they only say whether there is a result. That a function is *checked* to have no effect — no write, no global state, no input or output — goes back to Euclid (1977), a language designed for verifiable programs; Ada's functions with `in` parameters only, Fortran 95's `PURE` and Nim's `func` beside `proc` are the same idea.

`mutable` on a parameter that is written

> Ada's parameter modes (`in`, `out`, `in out`, 1983) are the first to make a routine say in its signature what it changes; Swift's `inout` and Rust's `&mut` are the modern forms. A field that cannot change unless it says so is ML's default, and Rust's.

`returns` and `throws`

> CLU (Barbara Liskov, 1975) wrote a procedure's result as `returns (int)` and its exceptions as `signals (…)` after the parameter list. `throws` is Java's word for the second.
>
> What happens underneath is Rust's. A routine that can fail returns a `Result[Payload, Error]` — a union of the value and the error — and nothing is unwound: Rust's `Result` gave that type its name and made it the one way a systems language reports failure, after Haskell's `Either` had shown that an error can be an ordinary value. And `try` descends from Rust's `try!` macro, the predecessor of its `?` operator: look at the result, go on with the value, or return the error to the caller at once — which Scaly spells `else throw`. Swift (2015) put the vocabulary of exceptions, `throws` and `try`, on the same mechanism, and

Scaly follows it in the words.

Unions and `choose`

A type that is one of several named alternatives, each with its own payload, taken apart by a construct that names the alternative and binds the payload: ALGOL 68's united modes and conformity clause are the first form, Hope (Edinburgh, 1980) and ML made it the centre of a language as algebraic data types with pattern matching, and Rust and Swift brought it to systems programming. The word `when` for an arm is Ada's.

`Option`, the `?`, and `null`

The null reference is Tony Hoare's, introduced into ALGOL W in 1965 and later called by him his billion-dollar mistake. The repair — a separate type for "a value or nothing" — is ML's `option`. The question mark after a type is C#'s (2005), and Swift and Kotlin made `T?` the only type that can hold `null`, as Scaly does. That an optional reference costs no extra space, the null address serving as the empty case, is known from Rust.

Generics in square brackets

CLU had parameterized types written `array[int]`; Eiffel and Scala kept the brackets where C++ and Java chose angle brackets and the parsing trouble that comes with them. Compiling one copy of the code per type argument is how Ada's generics and C++'s templates were always implemented. The names of those copies in the object file follow the Itanium C++ ABI, so that every existing debugger and linker tool can read them.

Lambdas and delegates

The anonymous function is Alonzo Church's lambda calculus (1930s) and Lisp's `LAMBDA`; the backslash as the nearest ASCII character to λ is Haskell's. `delegate` for the named type of a function value together with what it captured is C#'s.

`ref[T]` beside `pointer[T]`, `&` and `*`

The address-of and dereference operators are B's. The reference — an address that is used as if it were the thing itself and cannot do arithmetic — is C++'s (1985).

`is` and `as`

The type test written `v IS T` is Oberon's (Niklaus Wirth, 1987). The pair — `is` to ask for a type and `as` to convert to it, both written after the value — is Object Pascal's as Delphi had it (1995), and went with its designer, Anders Hejlsberg, into C# (2000). Swift, Kotlin and the Clinical Quality Language have the pair from there; Rust has `as`.

C.5 Memory

Scaly frees memory by region: every object belongs to a page that belongs to a call or to a longer-lived owner, and the whole region goes at once. Allocating by lifetime instead of by object is old systems practice — the arenas, zones and pools of compilers and servers, described by David Hanson in 1990. Making it a property of the *language*, with the compiler working out which region each value belongs to, is the work of Mads Tofte and Jean-Pierre Talpin (1994) and its implementation in the ML Kit. Cyclone (2002) brought regions to a C-like language and had the programmer name them; Rust's lifetimes descend from Cyclone.

Scaly stands closer to Tofte and Talpin than to Rust: the regions are inferred, ordinary code names none, and what the programmer sees is a refusal when a value would outlive its region. That check is escape analysis — the analysis Java and Go compilers use to decide what may live on the stack — turned from an optimization into a rule.

The pattern `0xDD` that `SCALY_POISON` writes over released memory is the one Microsoft's debug heap writes over freed blocks.

C.6 Self-Scaling

That a plain loop runs in parallel when the compiler can prove it safe, with no syntax to ask for it, is the promise of the auto-parallelizing Fortran compilers of the 1980s; Fortress (Guy Steele, Sun, 2006) made the parallel loop the default of a language, and ParaSail (Tucker Taft, 2009) built a whole language around implicit parallelism without a global heap. What they needed and what Scaly has in its checked `function` is a way to know that a body has no effects.

The engine underneath is Cilk's (MIT, 1994): fork-join tasks scheduled by work stealing. When a potential fork becomes a real one is decided by heartbeat scheduling (Acar, Charguéraud, Guatto, Rainey and Sieczkowski, 2018), which bounds the cost of parallelism against the work done.

Fibers are coroutines (Melvin Conway, 1963); the routine that switches between them is modelled on Boost.Context. Channels come from Tony Hoare's Communicating Sequential Processes (1978) by way of occam and Go, and from Go comes the principle that a routine doing input and output looks like any other and needs no `async` mark.

C.7 Modules and Packages

`module` and `use`

> The module as the unit that hides its insides is Modula's (Niklaus Wirth, 1975), and `use` to make another unit's names visible is Ada's. The system as Scaly has it is Rust's: a file declares its child modules by name (`module parser`, Rust's `mod parser;`) and the compiler finds the file; the modules form a tree of paths; `use` shortens a path to its last name or, with a star, opens a whole module; and a package has one root file from which

everything else is reached.

The interface beside the implementation

Modula-2's definition modules and Ada's package specifications separated what a unit promises from how it keeps the promise, and were written by hand. Scaly's are generated from the sources, as Swift's module interfaces are.

The prelude

A standard library that is visible without being asked for, under that name, is Haskell's.

A package is its sources

Go: packages are distributed as source and compiled into a cache on the machine that uses them.

C.8 An Unexpected Ancestor

The Clinical Quality Language is a language of the health-care standards body HL7 for stating clinical rules and quality measures — a pure expression language over patient data, and no relative of any systems language. Scaly's author worked with it, and it left marks:

- `define` as the one word that introduces a named thing. A CQL library is a list of `define` statements, each a name and what it stands for.
- The logical operators as words, including `xor`; CQL writes `and`, `or`, `not`, `xor` and has no symbols for them.
- `is` and `as` after a value, to test its type and to convert it — which CQL in turn has from C#.
- A function's result type after the word `returns`.
- An identifier in quotes, which may then contain spaces or any other character.
- The version of a library named where the library is asked for, in the source text itself and not in a file beside it.
- = for equality in a language whose definitions have no need of it for anything else.

C.9 And What Is Scaly's Own?

Less than one would like to claim. The honest account of four candidates:

One `define` for everything

> A structure, a union, a namespace, a constant, a type alias and a delegate type are all a `define` followed by what the name stands for; a structure without fields *is* a namespace. The ambition is not new: BETA (Kristensen, Madsen, Møller-Pedersen and Nygaard, from 1976) folded class, procedure, function and type into a single construct, the pattern, and Zig binds structures, unions and namespaces alike with an ordinary constant declaration. Scaly's form — one keyword, and the shape of what follows decides — was arrived at on its own, and no other language is known to draw the line in the same place.

Precedence is not part of the grammar

> Not Scaly's invention. Prolog let a program declare its operators and their priorities in the 1970s, ML and Haskell have fixity declarations, Swift has precedence groups; all of them parse an expression as a flat sequence first and give it structure afterwards, as Scaly does.

The operation, read strictly from left to right

> Smalltalk reads its binary messages from left to right with no precedence at all, APL from right to left, and Pony refuses to choose and demands parentheses. What Scaly is building goes one step further: the *type* of the value so far says what the next thing means — an argument, a member, an operator that leaves a value waiting for its right side, as a section does in Haskell. That combination has no model that the author knows of. It is also not finished: the binding strength of the operators is such a protocol of waiting types already, but a step is still chosen by the shape of what follows as well as by the type.

No semicolons

> The line end as a separator is Fortran's and BASIC's, and Python, Go, Swift, Kotlin and Scala all found ways to spare the programmer the semicolon. Scaly's part is small: the line end and the colon are the same token, so that the grammar needs no rule about where a statement may continue, and the semicolon, no longer wanted, became the comment sign.

One thing is a discipline more than an invention, and the author held to it throughout: *every sign means one thing.* Parentheses are always a tuple — a parameter list is a tuple, the fields of a structure are a tuple, the arguments of a call are a tuple, and because a tuple of one is that one, a single argument needs no parentheses at all. Braces are always a block. Brackets are always a list, of values or of types. And no arithmetic sign is borrowed for something that is not arithmetic: < and > compare and never enclose type arguments, = compares and never assigns, and the signs that are not arithmetic — `#`, `^`, `!`, `?`, the colon, the backslash — are kept out of operators altogether. Most languages descended from C give up each of these somewhere: parentheses that group, call and cast, braces that are blocks here and values there, angle brackets that are generics until they are a shift. The exceptions Scaly allows itself are inherited ones — the = inside ALGOL's `:=`, and B's prefix `*` and `&` for following and taking an address.

What does seem to be Scaly's own lies elsewhere. The lifetime suffixes on a construction — `#` for the caller's region, `^name` for a named one, `!` for a thrown value — are a notation found nowhere else, small as it is. So is the pairing that gives the language its name: regions the compiler infers, functions the compiler checks to be pure, and a plain `for` that runs in parallel because of both. Each part has a history told above; that one language rests all three on each other is new. And no other compiler is known whose grammar is an SGML document.

C.10 The Implementation

A compiler written in its own language

> The Lisp compiler of Timothy Hart and Michael Levin (1962) is the first that compiled itself. Shipping the compiler in a portable intermediate form, so that a new machine can build it without already having it, is the Pascal-P kit from Zurich (1970s); Scaly's seed is the same idea with LLVM's textual IR in place of P-code.

LLVM

> Everything behind the emitter — optimization, code generation for every target, the in-process JIT — is LLVM, begun by Chris Lattner and Vikram Adve at the University of Illinois in 2000. Debug information is DWARF.

The grammar as a document

> Scaly's grammar is an SGML document (Charles Goldfarb; ISO 8879, 1986), and parser, syntax tree and editor grammar are generated from it by DSSSL stylesheets (ISO/IEC 10179, 1996). The tools were James Clark's SP and Jade, and are now ports of them written in Scaly. This book is DocBook, produced by the same route.

Tests inside the prose

> The chapters on expressions, definitions and control flow are generated from documents whose examples are executed as tests. That a program and its explanation are one text is Donald Knuth's literate programming (1984).

The REPL and the language server

> The read-eval-print loop is Lisp's, in name and in substance. The language server speaks the protocol Microsoft published in 2016 for Visual Studio Code.